Daily french bakery sales

In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns

from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import adfuller, kpss, acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import ccf
from statsmodels.stats.outliers_influence import variance_inflation_facto

import lightgbm as lgb

from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.preprocessing import LabelEncoder

import warnings
warnings.filterwarnings("ignore")

import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
```

Import Data and set it as a timeseries

In [2]:
```
df = pd.read_csv('./data/daily_sales_french_bakery.csv',)
df['ds'] = pd.to_datetime(df['ds'], format='%Y-%m-%d')
# df = df.set_index('ds')
```

Check Data

In [3]:
```
df
```

Out[3]:

	unique_id	ds	y	unit_price
0	12 MACARON	2022-07-13	10.0	10.0
1	12 MACARON	2022-07-14	0.0	10.0
2	12 MACARON	2022-07-15	0.0	10.0
3	12 MACARON	2022-07-16	10.0	10.0
4	12 MACARON	2022-07-17	30.0	10.0
...	...	...	...	...
57041	VIK BREAD	2022-09-26	0.0	2.7
57042	VIK BREAD	2022-09-27	0.0	2.7
57043	VIK BREAD	2022-09-28	0.0	2.7
57044	VIK BREAD	2022-09-29	0.0	2.7
57045	VIK BREAD	2022-09-30	32.4	2.7

57046 rows × 4 columns

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 57046 entries, 0 to 57045
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   unique_id   57046 non-null  object
 1   ds          57046 non-null  datetime64[ns]
 2   y           57046 non-null  float64
 3   unit_price  57046 non-null  float64
dtypes: datetime64[ns](1), float64(2), object(1)
memory usage: 1.7+ MB
```

In [5]:
```
df.describe()
```

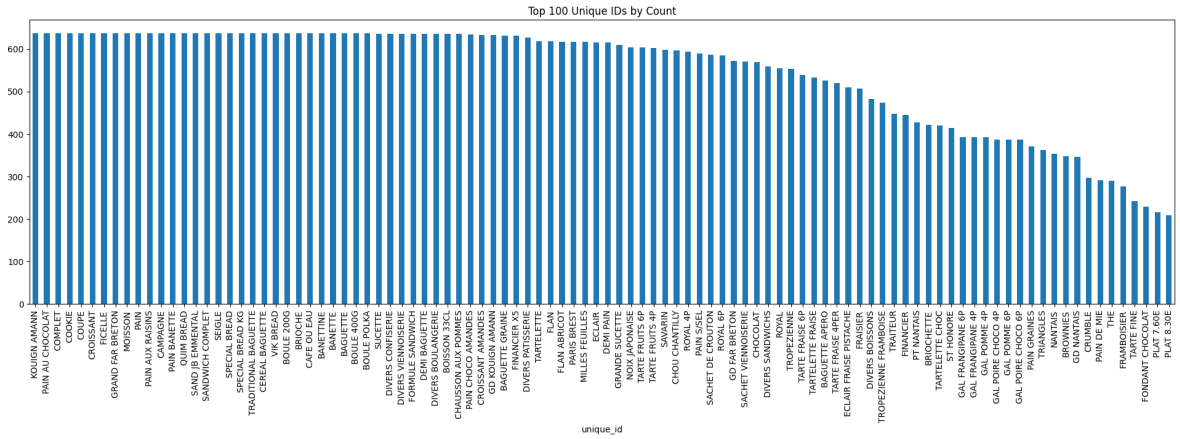
Out[5]:

	ds	y	unit_price
count	57046	57046.000000	57046.000000
mean	2021-11-18 13:24:30.770956544	9.792814	3.631011
min	2021-01-02 00:00:00	0.000000	0.000000
25%	2021-06-25 00:00:00	0.000000	1.400000
50%	2021-11-17 00:00:00	0.900000	2.000000
75%	2022-04-17 00:00:00	8.000000	4.500000
max	2022-09-30 00:00:00	919.100000	60.000000
std	NaN	32.327068	3.909706

In [6]:
```
fig,ax = plt.subplots(figsize=(24,6))


df['unique_id'].value_counts().head(100).plot(kind='bar',ax=ax)
```

```
ax.set_title('Top 100 Unique IDs by Count')
```

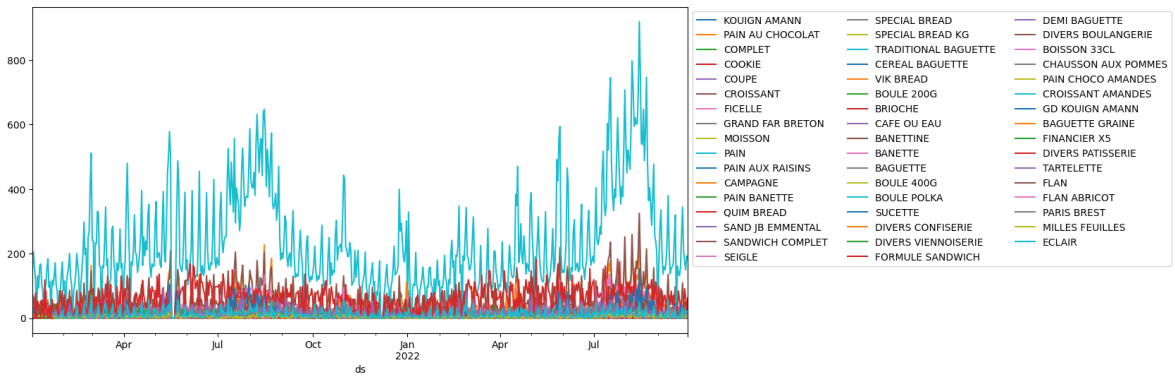
Out[6]: Text(0.5, 1.0, 'Top 100 Unique IDs by Count')



In [7]:
```
fig,ax = plt.subplots(figsize=(12,6))


for unique_id in df['unique_id'].value_counts().head(50).index:
    print(unique_id)
    mask =  (df['unique_id']==unique_id)
    df[mask].plot(x='ds',y='y',ax=ax,label=unique_id)

ax.legend(loc='upper left', bbox_to_anchor=(1, 1),ncol=3)
# df[df['unique_id']=='COOKIE'].plot(x='ds',y='y',ax=ax,label='COOKIE')
# df[df['unique_id']=='BAGUETTE'].plot(x='ds',y='y',ax=ax,label='BAGUETTE
# df[df['unique_id']=='PAIN AU CHOCOLAT'].plot(x='ds',y='y',ax=ax,label='
```

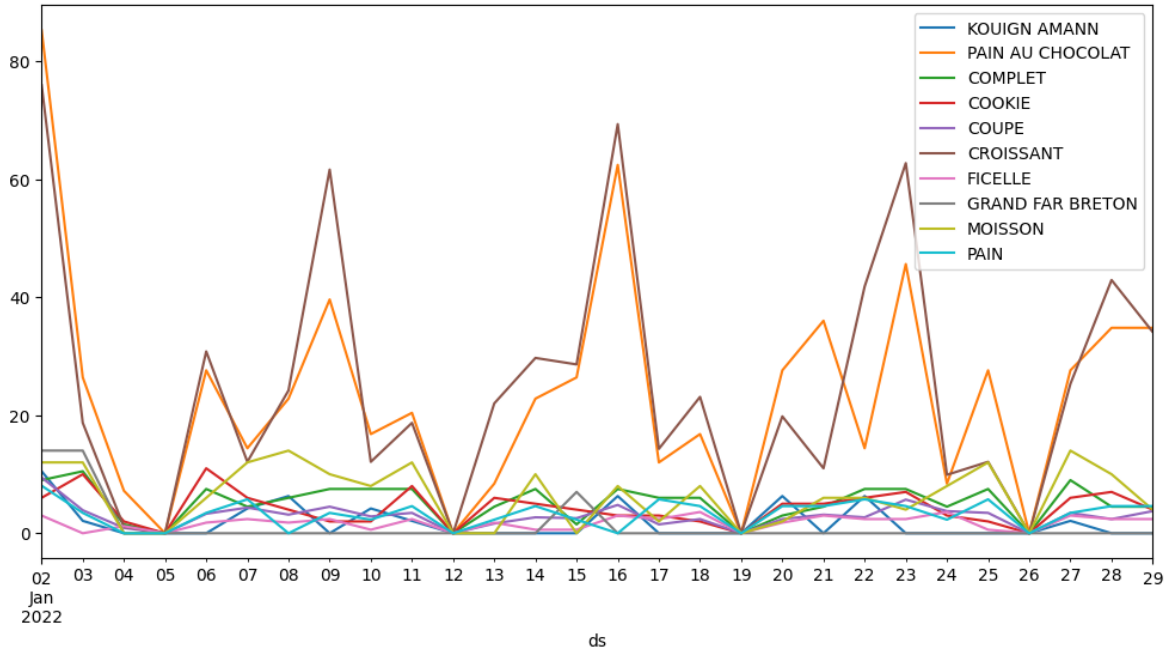
```
KOUIGN AMANN
PAIN AU CHOCOLAT
COMPLET
COOKIE
COUPE
CROISSANT
FICELLE
GRAND FAR BRETON
MOISSON
PAIN
PAIN AUX RAISINS
CAMPAGNE
PAIN BANETTE
QUIM BREAD
SAND JB EMMENTAL
SANDWICH COMPLET
SEIGLE
SPECIAL BREAD
SPECIAL BREAD KG
TRADITIONAL BAGUETTE
CEREAL BAGUETTE
VIK BREAD
BOULE 200G
BRIOCHE
CAFE OU EAU
BANETTINE
BANETTE
BAGUETTE
BOULE 400G
BOULE POLKA
SUCETTE
DIVERS CONFISERIE
DIVERS VIENNOISERIE
FORMULE SANDWICH
DEMI BAGUETTE
DIVERS BOULANGERIE
BOISSON 33CL
CHAUSSON AUX POMMES
PAIN CHOCO AMANDES
CROISSANT AMANDES
GD KOUIGN AMANN
BAGUETTE GRAINE
FINANCIER X5
DIVERS PATISSERIE
TARTELETTE
FLAN
FLAN ABRICOT
PARIS BREST
MILLES FEUILLES
ECLAIR
```

Out[7]: <matplotlib.legend.Legend at 0x75216f131280>



In [8]:
```
fig,ax = plt.subplots(figsize=(12,6))


for unique_id in df['unique_id'].value_counts().head(10).index:
    print(unique_id)
    mask = (df['ds'] > '2022-01-01') & (df['ds'] < '2022-01-30') & (df['u
    df[mask].plot(x='ds',y='y',ax=ax,label=unique_id)

# df[df['unique_id']=='COOKIE'].plot(x='ds',y='y',ax=ax,label='COOKIE')
# df[df['unique_id']=='BAGUETTE'].plot(x='ds',y='y',ax=ax,label='BAGUETTE
# df[df['unique_id']=='PAIN AU CHOCOLAT'].plot(x='ds',y='y',ax=ax,label='
```

```
KOUIGN AMANN
PAIN AU CHOCOLAT
COMPLET
COOKIE
COUPE
CROISSANT
FICELLE
GRAND FAR BRETON
MOISSON
PAIN
```

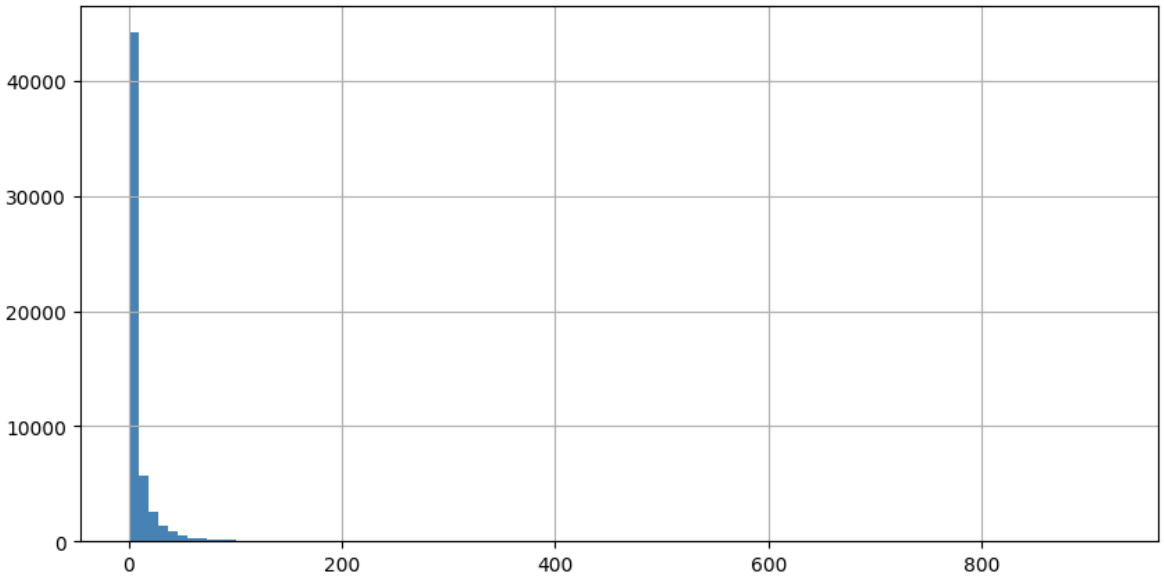


Check outliers

In [ ]:

In [9]:
```
df['y'].hist(bins=100, figsize=(10,5), color='steelblue')
```

Out[9]: <Axes: >



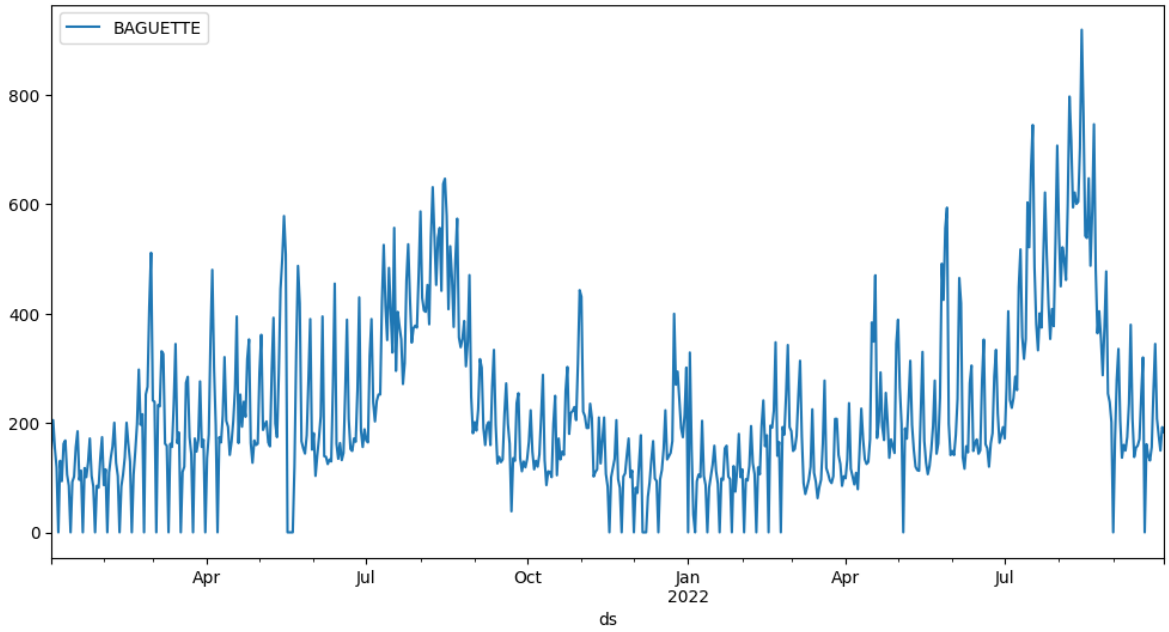
In [10]:
```
df[df['y']>600]
```

Out[10]:

	unique_id	ds	y	unit_price
54092	TRADITIONAL BAGUETTE	2021-08-08	631.2	1.2
54098	TRADITIONAL BAGUETTE	2021-08-14	637.2	1.2
54099	TRADITIONAL BAGUETTE	2021-08-15	646.8	1.2
54432	TRADITIONAL BAGUETTE	2022-07-14	603.2	1.3
54434	TRADITIONAL BAGUETTE	2022-07-16	664.3	1.3
54435	TRADITIONAL BAGUETTE	2022-07-17	744.9	1.3
54442	TRADITIONAL BAGUETTE	2022-07-24	621.4	1.3
54449	TRADITIONAL BAGUETTE	2022-07-31	707.2	1.3
54456	TRADITIONAL BAGUETTE	2022-08-07	796.9	1.3
54457	TRADITIONAL BAGUETTE	2022-08-08	716.3	1.3
54459	TRADITIONAL BAGUETTE	2022-08-10	621.4	1.3
54460	TRADITIONAL BAGUETTE	2022-08-11	600.6	1.3
54461	TRADITIONAL BAGUETTE	2022-08-12	604.5	1.3
54462	TRADITIONAL BAGUETTE	2022-08-13	703.3	1.3
54463	TRADITIONAL BAGUETTE	2022-08-14	919.1	1.3
54464	TRADITIONAL BAGUETTE	2022-08-15	760.5	1.3
54467	TRADITIONAL BAGUETTE	2022-08-18	647.4	1.3
54470	TRADITIONAL BAGUETTE	2022-08-21	746.2	1.3

In [11]:
```
fig,ax = plt.subplots(figsize=(12,6))
df[df["unique_id"] == "TRADITIONAL BAGUETTE"].plot(x='ds',y='y',ax=ax,lab
```

Out[11]: <Axes: xlabel='ds'>



Basically it does not seem like these are outliers

In [ ]:

Data cleaning

Rename Columns

In [12]:
```
df.columns
```

Out[12]: Index(['unique_id', 'ds', 'y', 'unit_price'], dtype='object')

In [ ]:
```
df = df.rename(columns = {'ds': 'Date', 'y': 'Sales', 'unique_id': 'Produ
```

In [14]:
```
df
```

Out[14]:

	Product	Date	Sales	unit_price
0	12 MACARON	2022-07-13	10.0	10.0
1	12 MACARON	2022-07-14	0.0	10.0
2	12 MACARON	2022-07-15	0.0	10.0
3	12 MACARON	2022-07-16	10.0	10.0
4	12 MACARON	2022-07-17	30.0	10.0
...	...	...	...	...
57041	VIK BREAD	2022-09-26	0.0	2.7
57042	VIK BREAD	2022-09-27	0.0	2.7
57043	VIK BREAD	2022-09-28	0.0	2.7
57044	VIK BREAD	2022-09-29	0.0	2.7
57045	VIK BREAD	2022-09-30	32.4	2.7

57046 rows × 4 columns

Make Time the index

In [15]:
```
df = df.set_index('Date',drop=False)
df = df.rename_axis('Index_Date')
df
```

Out[15]:

	Product	Date	Sales	unit_price
Index_Date				
2022-07-13	12 MACARON	2022-07-13	10.0	10.0
2022-07-14	12 MACARON	2022-07-14	0.0	10.0
2022-07-15	12 MACARON	2022-07-15	0.0	10.0
2022-07-16	12 MACARON	2022-07-16	10.0	10.0
2022-07-17	12 MACARON	2022-07-17	30.0	10.0
...	...	...	...	...
2022-09-26	VIK BREAD	2022-09-26	0.0	2.7
2022-09-27	VIK BREAD	2022-09-27	0.0	2.7
2022-09-28	VIK BREAD	2022-09-28	0.0	2.7
2022-09-29	VIK BREAD	2022-09-29	0.0	2.7
2022-09-30	VIK BREAD	2022-09-30	32.4	2.7

57046 rows × 4 columns

Sort it in a nice way

In [16]:
```
df = df.sort_values(by=['Product', 'Date']).copy()
```

```
df
```

Out[16]:

	Product	Date	Sales	unit_price
Index_Date				
2022-07-13	12 MACARON	2022-07-13	10.0	10.0
2022-07-14	12 MACARON	2022-07-14	0.0	10.0
2022-07-15	12 MACARON	2022-07-15	0.0	10.0
2022-07-16	12 MACARON	2022-07-16	10.0	10.0
2022-07-17	12 MACARON	2022-07-17	30.0	10.0
...	...	...	...	...
2022-09-26	VIK BREAD	2022-09-26	0.0	2.7
2022-09-27	VIK BREAD	2022-09-27	0.0	2.7
2022-09-28	VIK BREAD	2022-09-28	0.0	2.7
2022-09-29	VIK BREAD	2022-09-29	0.0	2.7
2022-09-30	VIK BREAD	2022-09-30	32.4	2.7

57046 rows × 4 columns

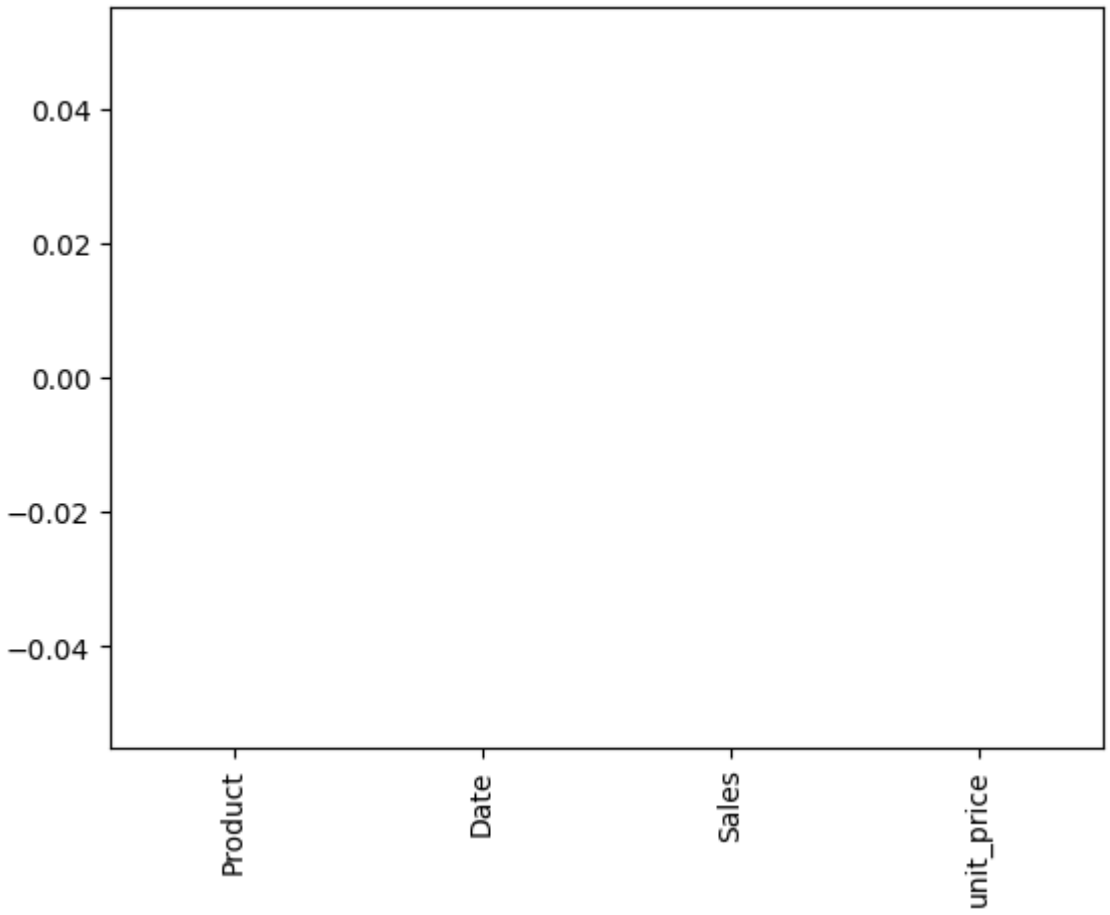
Deal with missing values (if any)

In [17]:

```
df.isna().sum().plot(kind='bar')
df.isna().sum()
```

Out[17]:

```
Product       0
Date          0
Sales         0
unit_price    0
dtype: int64
```



(NOT HERE) Widen the matrix if not many exogenous variables

In [18]:
```
# # 1. Pivot: Dates become the index, Products become column headers
# df_wide = df.pivot(index='Date', columns='Product', values='Sales')
```

Make sure you have rows for all dates in the range, even if sales are zero

This only works if you have one row per day, here it does not so skip it. You would have to widen the dataframe (each product becomes a column) to do this, but it messes up with the exogenic variables

In [19]:
```
# df.info()
```

In [20]:
```
# df
```

In [21]:
```
# # 1. Pivot: Dates become the index, Products become column headers
# df_wide = df.pivot(index='Date', columns='Product', values='Sales')

# # 2. Now asfreq('D') will work perfectly because the index is unique
# df_wide = df_wide.asfreq('D')

# # 3. Fill missing days with 0 (since no rows means 0 sales)
# df_wide = df_wide.fillna(0)
```

In [22]:
```
# df_wide
```

Here there is no missing values, but if there were we could chose two ways to handle them:

Forward fill (ffill)

Generally the best to prevent data leakage

In [179…
```
# Use transform to keep the original index structure
df['Sales'] = df.groupby('Product')['Sales'].ffill()

# Optional: If the VERY first value is NaN, ffill has nothing to grab.
# You might want to backfill (bfill) those specific cases.
df['Sales'] = df.groupby('Product')['Sales'].bfill()
```

Linear interpolation fill

If you use Linear Interpolation for your training set, be careful! Standard interpolation uses future values to fill past NaNs (it looks ahead to see where the line should end).

While this is fine for historical analysis, if you use it on the very end of your dataset to "prepare" it for a LightGBM prediction, you are essentially giving the model a sneak peek at the future. For real-time machine learning, ffill is almost always the safer bet because it only uses information that was available at that specific moment in time.

In [144…
```
df['Sales'] = df.groupby('Product')['Sales'].transform(lambda x: x.interp

# Optional: If the VERY first value is NaN, ffill has nothing to grab.
# You might want to backfill (bfill) those specific cases.
df['Sales'] = df.groupby('Product')['Sales'].bfill()
```

Fill with zeros

In [23]:
```
df['Sales'] = df['Sales'].fillna(0)
```

Data Exploration

This is focusing especially on one of the products

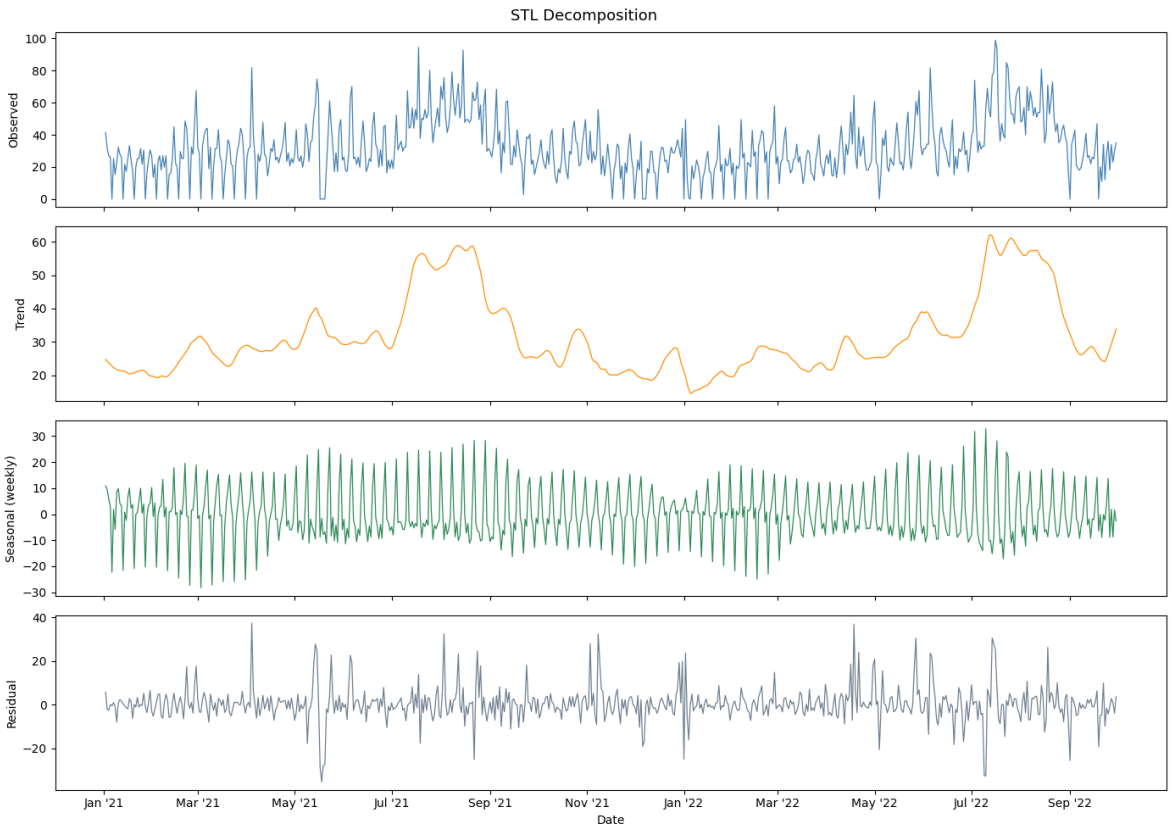
STL Decomposition

See if there is seasonality in the data.

STL (Seasonal-Trend decomposition using Loess) is a time series decomposition method that separates data into three components: trend, seasonality, and residual (noise). It uses locally weighted regression (Loess) to flexibly estimate smooth patterns, making it robust to non-linear trends and changing seasonal effects. STL is widely used because it

handles complex real-world data well and allows users to control the smoothness of each component.

In [24]:
```
## Make the seasonality plot



# Work on store_1 for the decomposition (fill NaN temporarily for STL)
# s1 = (df_raw[df_raw["store"] == "store_1"]
#       .set_index("date")["sales"]
#       .sort_index()
#       .interpolate("time"))   # temporary linear fill for STL only

df_baguette = df[df['Product'] == 'BAGUETTE'].copy()

stl    = STL(df_baguette['Sales'], period=7, robust=True)
result = stl.fit()

fig, axes = plt.subplots(4, 1, figsize=(14, 10), sharex=True)
for ax, data, label, color in zip(
    axes,
    [df_baguette['Sales'], result.trend, result.seasonal, result.resid],
    ["Observed", "Trend", "Seasonal (weekly)", "Residual"],
    ["steelblue", "darkorange", "seagreen", "slategray"]
):
    ax.plot(data, lw=0.9, color=color)
    ax.set_ylabel(label, fontsize=10)
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b '%y"))

axes[-1].set_xlabel("Date")
fig.suptitle("STL Decomposition ", fontsize=13)
plt.tight_layout()
plt.show()

# Strength of trend & seasonality
var_resid    = np.var(result.resid)
var_deseason = np.var(result.trend + result.resid)
var_detrend  = np.var(result.seasonal + result.resid)
F_trend      = max(0, 1 - var_resid / var_deseason)
F_seasonal   = max(0, 1 - var_resid / var_detrend)
print(f"Trend strength     : {F_trend:.3f}  (0=none, 1=perfect)")
print(f"Seasonal strength  : {F_seasonal:.3f}  (0=none, 1=perfect)")
```




```
Trend strength     : 0.692  (0=none, 1=perfect)
Seasonal strength  : 0.606  (0=none, 1=perfect)
```

Stationarity test

Most classical models (SARIMAX) require stationarity. Even for LightGBM, non-stationarity inflates lag feature importance artificially. Run two tests:

- ADF (Augmented Dickey-Fuller): null hypothesis is non-stationary. A low p-value (< 0.05) means stationarity. statsmodels.tsa.stattools.adfuller
- KPSS: null hypothesis is stationary. A low p-value means non-stationarity. Use both together — they're complementary.

In [25]:
```
df_baguette.index = df_baguette['Date']
```

In [26]:
```
def adf_kpss_report(series, label=""):
    series = series.dropna()
    adf_stat, adf_p, _, _, adf_cv, _ = adfuller(series, autolag="AIC")
    try:
        kpss_stat, kpss_p, _, kpss_cv = kpss(series, regression="c", nlag
    except Exception:
        kpss_stat, kpss_p = np.nan, np.nan

    print(f"{'—'*50}")
    print(f"{label}")
    print(f"  ADF  stat={adf_stat:.3f}  p={adf_p:.4f}  "
          f"→ {'STATIONARY' if adf_p < 0.05 else 'NON-STATIONARY'}")
    print(f"  KPSS stat={kpss_stat:.3f}  p={kpss_p:.4f}  "
          f"→ {'STATIONARY' if kpss_p > 0.05 else 'NON-STATIONARY'}")
```

```python
# Raw series
adf_kpss_report(df_baguette['Sales'], "Raw series (baguette)")

# First difference
df_baguette_diff = df_baguette['Sales'].diff().dropna()
adf_kpss_report(df_baguette_diff, "First difference")

# Seasonal difference (period=7)
df_baguette_seas_diff = df_baguette['Sales'].diff(7).dropna()
adf_kpss_report(df_baguette_seas_diff, "Seasonal difference (lag-7)")
```

```
────────────────────────────────────────────────────────
Raw series (baguette)
  ADF  stat=-2.458  p=0.1261  → NON-STATIONARY
  KPSS stat=0.385  p=0.0836  → STATIONARY
────────────────────────────────────────────────────────
First difference
  ADF  stat=-6.587  p=0.0000  → STATIONARY
  KPSS stat=0.038  p=0.1000  → STATIONARY
────────────────────────────────────────────────────────
Seasonal difference (lag-7)
  ADF  stat=-6.714  p=0.0000  → STATIONARY
  KPSS stat=0.041  p=0.1000  → STATIONARY
```

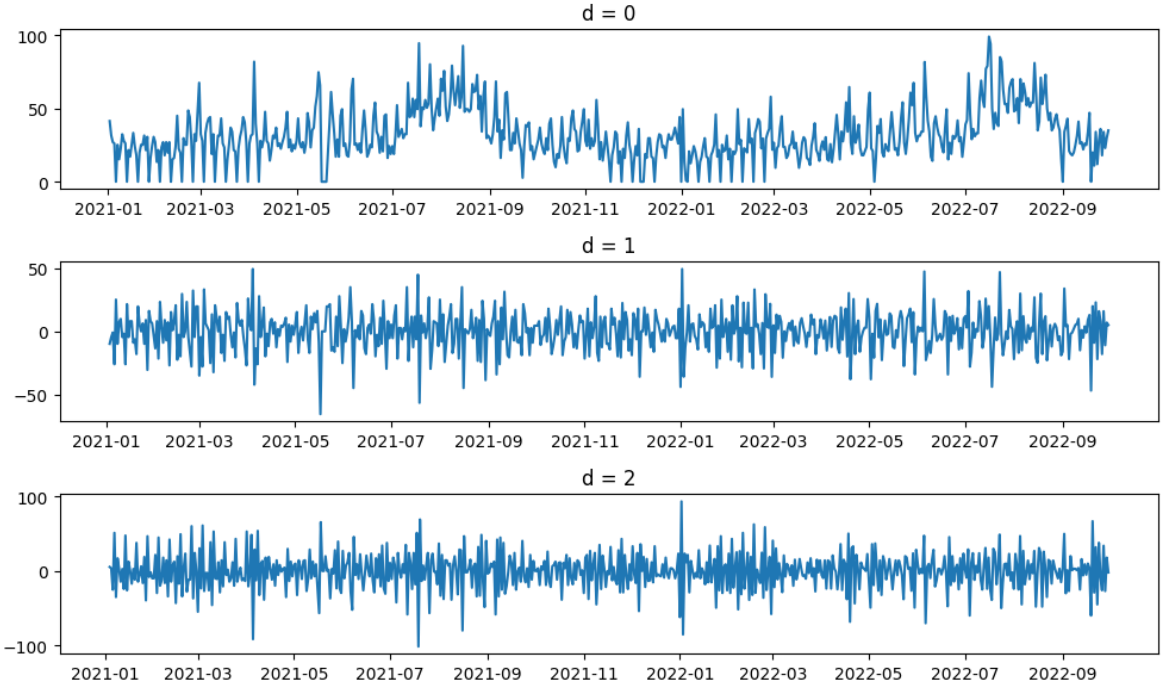
Verify I degree (d : differencing order) for ARIMA

d should be the first one for which it looks stationary

In [27]:

```python
# Function to difference the series
def difference(series, d):
    diff_series = series.copy()
    for _ in range(d):
        diff_series = diff_series.diff()
    return diff_series

# Try different d values
d_values = [0, 1, 2]

plt.figure(figsize=(10, 6))

for i, d in enumerate(d_values, 1):
    plt.subplot(len(d_values), 1, i)
    plt.plot(difference(df_baguette['Sales'], d))
    adf_kpss_report(difference(df_baguette['Sales'], d), label=f"d = {d}"
    plt.title(f"d = {d}")
    plt.tight_layout()

plt.show()
```

```
──────────────────────────────────────────────────────────
d = 0
  ADF  stat=-2.458  p=0.1261  → NON-STATIONARY
  KPSS stat=0.385  p=0.0836  → STATIONARY
──────────────────────────────────────────────────────────
d = 1
  ADF  stat=-6.587  p=0.0000  → STATIONARY
  KPSS stat=0.038  p=0.1000  → STATIONARY
──────────────────────────────────────────────────────────
d = 2
  ADF  stat=-10.911  p=0.0000  → STATIONARY
  KPSS stat=0.036  p=0.1000  → STATIONARY
```



ACF and PACF plots

The ACF (autocorrelation function) and PACF (partial autocorrelation function) plots help identify the structure of a time series for models like ARIMA.

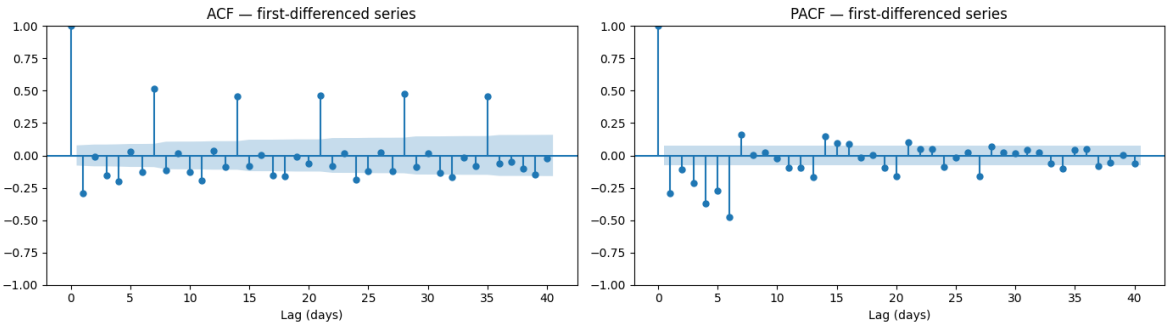
The ACF shows how observations are correlated with past values at different lags, which is useful for detecting the moving average (MA) order (q): a sharp cutoff in the ACF suggests the appropriate (q).

The PACF measures the direct correlation between observations at a given lag after removing intermediate effects, helping identify the autoregressive (AR) order (p): a sharp cutoff in the PACF indicates the appropriate (p).

If both ACF and PACF decay gradually, it may indicate a mixed ARMA model. Differencing ((d)) is determined separately by checking stationarity (e.g., trends or unit roots).

In [28]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

plot_acf(df_baguette_diff,  lags=40, ax=axes[0], title="ACF — first-diffe
plot_pacf(df_baguette_diff, lags=40, ax=axes[1], title="PACF — first-diff
```

```
for ax in axes:
    ax.set_xlabel("Lag (days)")
plt.tight_layout()
plt.show()

print("Interpretation guide:")
print("  ACF  : if it cuts off after lag q  → MA(q) component")
print("  PACF : if it cuts off after lag p  → AR(p) component")
print("  Spikes at multiples of 7 → residual weekly seasonality")
```

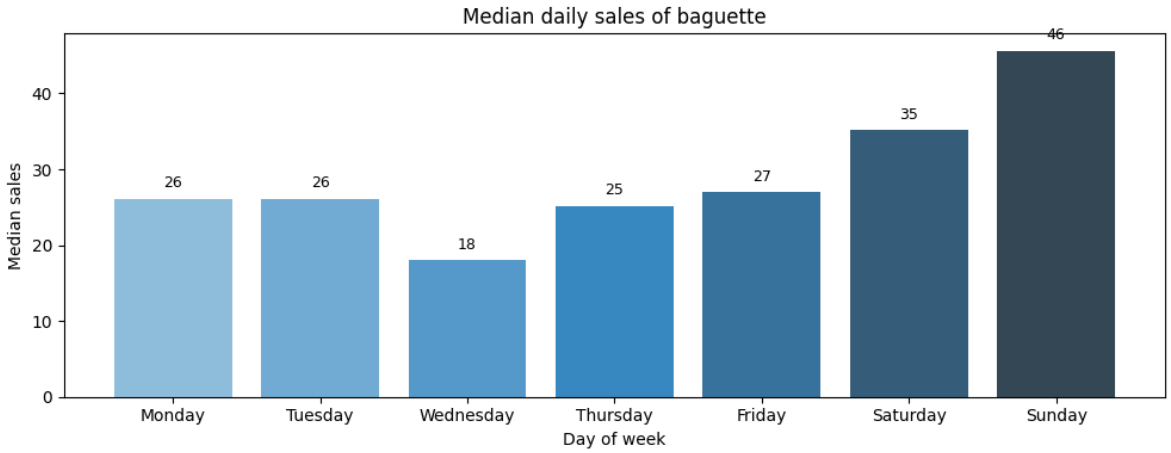



```
Interpretation guide:
  ACF  : if it cuts off after lag q  → MA(q) component
  PACF : if it cuts off after lag p  → AR(p) component
  Spikes at multiples of 7 → residual weekly seasonality
```

Seasonality analysis

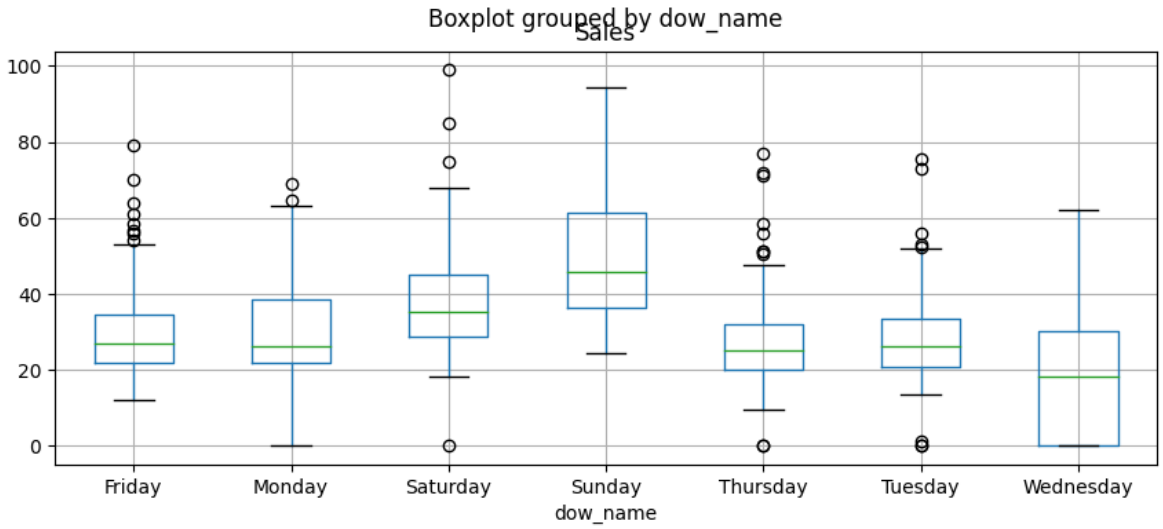
Plot a seasonal subseries plot (average value by month/day-of-week/hour). This shows you the shape of seasonal patterns clearly. Use periodogram or FFT to detect non-obvious periodicities in the frequency domain.

In [29]:
```
# s1_full = df_raw[df_raw["store"] == "store_1"].copy()
# s1_full["dow"]      = s1_full["date"].dt.dayofweek
# s1_full["dow_name"] = s1_full["date"].dt.day_name()

df_baguette_full = df[df['Product'] == 'BAGUETTE'].copy()
df_baguette_full["dow"]      = df_baguette_full["Date"].dt.dayofweek
df_baguette_full["dow_name"] = df_baguette_full["Date"].dt.day_name()


dow_order  = ["Monday","Tuesday","Wednesday","Thursday","Friday","Saturda
dow_median = df_baguette_full.groupby("dow_name")["Sales"].median().reind

fig, ax = plt.subplots(figsize=(10, 4))
bars = ax.bar(dow_order, dow_median, color=sns.color_palette("Blues_d", 7
ax.set_title("Median daily sales of baguette", fontsize=12)
ax.set_xlabel("Day of week")
ax.set_ylabel("Median sales")
for bar, val in zip(bars, dow_median):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
            f"{val:.0f}", ha="center", va="bottom", fontsize=9)
plt.tight_layout()
plt.show()
```



In [30]:
```
df_baguette_full.boxplot("Sales",by="dow_name", figsize=(10,4))
```

Out[30]: <Axes: title={'center': 'Sales'}, xlabel='dow_name'>



In [ ]:

Cross correlation

In [31]:
```
# 1. Prepare the Data
# Assuming 'df' has columns: ['Date', 'Product', 'Sales']
df['Date'] = pd.to_datetime(df['Date'])

# Pivot so each column is a product's time series
pivot_df = df.pivot(index='Date', columns='Product', values='Sales').fill

# IMPORTANT: Make data stationary (Differencing)
# This removes trends so we see the relationship between 'changes' in sal
pivot_df_diff = pivot_df.diff().dropna()

target_product = "BAGUETTE"
max_lags = 14  # Looking at the last 2 weeks of influence
results = []

# 2. Calculate CCF for each product vs Target
for col in pivot_df_diff.columns:
    if col == target_product:
```

```
        continue

    # ccf(x, y) measures how much x leads y
    # We want to see if 'col' leads 'target_product'
    correlation_values = ccf(pivot_df_diff[col], pivot_df_diff[target_pro

    # We only care about positive lags (past of col affecting future of t
    for lag in range(1, max_lags + 1):
        results.append({
            'Product': col,
            'Lag': lag,
            'Correlation': correlation_values[lag]
        })

# 3. Analyze Results
ccf_results = pd.DataFrame(results)

# Find the most relevant product-lag combinations
top_features = ccf_results.sort_values(by='Correlation', ascending=False)
print("Top Predictive Lags:")
print(top_features)
```

```
Top Predictive Lags:
                   Product  Lag  Correlation
1938  TRADITIONAL BAGUETTE    7     0.535748
1945  TRADITIONAL BAGUETTE   14     0.519561
433               CROISSANT   14     0.513193
426               CROISSANT    7     0.511472
1168      PAIN AU CHOCOLAT    7     0.486992
1175      PAIN AU CHOCOLAT   14     0.476160
419                   COUPE   14     0.453586
1868             TARTELETTE    7     0.442918
412                   COUPE    7     0.439961
76                 BANETTE    7     0.437981
622                 ECLAIR    7     0.430833
1749         SPECIAL BREAD   14     0.430644
1875             TARTELETTE   14     0.401595
629                 ECLAIR   14     0.395829
1742         SPECIAL BREAD    7     0.395682
69         BAGUETTE GRAINE   14     0.393105
307               CAMPAGNE   14     0.392320
62         BAGUETTE GRAINE    7     0.392203
1714               SAVARIN    7     0.389468
664                FICELLE    7     0.383199
1329           PARIS BREST   14     0.380604
132             BOULE 200G    7     0.376566
83                 BANETTE   14     0.375327
1322           PARIS BREST    7     0.374963
1721               SAVARIN   14     0.360698
300               CAMPAGNE    7     0.360600
146             BOULE 400G    7     0.358292
139             BOULE 200G   14     0.358135
1847       TARTE FRUITS 4P   14     0.345168
335        CEREAL BAGUETTE   14     0.335898
181                BRIOCHE   14     0.331479
174                BRIOCHE    7     0.327081
328        CEREAL BAGUETTE    7     0.326349
153             BOULE 400G   14     0.323502
1840       TARTE FRUITS 4P    7     0.318409
671                FICELLE   14     0.315256
790               FRAISIER    7     0.313831
370         CHOU CHANTILLY    7     0.312067
1189      PAIN AUX RAISINS   14     0.309097
97               BANETTINE   14     0.302430
1196           PAIN BANETTE    7     0.301675
1126        NOIX JAPONAISE    7     0.298165
1028          KOUIGN AMANN    7     0.296368
1091               MOISSON   14     0.295717
267            CAFE OU EAU    2     0.288316
1854       TARTE FRUITS 6P    7     0.288120
90               BANETTINE    7     0.288015
1077       MILLES FEUILLES   14     0.287980
1182      PAIN AUX RAISINS    7     0.284843
1133        NOIX JAPONAISE   14     0.283678
```

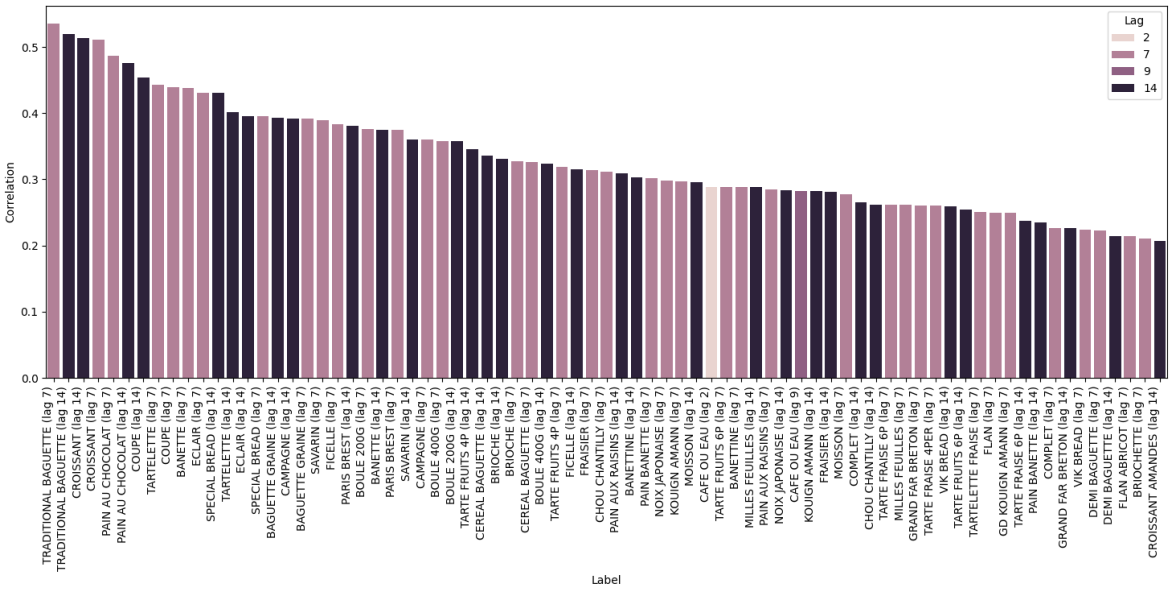
In [32]:

```
fig,ax = plt.subplots(figsize=(18,6))

top_features = ccf_results.sort_values(by='Correlation', ascending=False)

top_features['Label'] = top_features.apply(lambda row: f"{row['Product']}
sns.barplot(data=top_features, x='Label', y='Correlation', hue='Lag', ax=
# top_features.plot(kind='bar', x='Product', y='Correlation', figsize=(12
```

```
ax.set_xticklabels(ax.get_xticklabels(), rotation=90, ha='right', fontsiz

# xlabels = [f"{row['Product']} (lag {row['Lag']})" for _, row in top_fea
# ax.set_xticklabels(xlabels, rotation=90, ha='right', fontsize=10)
```

```
Out[32]: [Text(0, 0, 'TRADITIONAL BAGUETTE (lag 7)'),
          Text(1, 0, 'TRADITIONAL BAGUETTE (lag 14)'),
          Text(2, 0, 'CROISSANT (lag 14)'),
          Text(3, 0, 'CROISSANT (lag 7)'),
          Text(4, 0, 'PAIN AU CHOCOLAT (lag 7)'),
          Text(5, 0, 'PAIN AU CHOCOLAT (lag 14)'),
          Text(6, 0, 'COUPE (lag 14)'),
          Text(7, 0, 'TARTELETTE (lag 7)'),
          Text(8, 0, 'COUPE (lag 7)'),
          Text(9, 0, 'BANETTE (lag 7)'),
          Text(10, 0, 'ECLAIR (lag 7)'),
          Text(11, 0, 'SPECIAL BREAD (lag 14)'),
          Text(12, 0, 'TARTELETTE (lag 14)'),
          Text(13, 0, 'ECLAIR (lag 14)'),
          Text(14, 0, 'SPECIAL BREAD (lag 7)'),
          Text(15, 0, 'BAGUETTE GRAINE (lag 14)'),
          Text(16, 0, 'CAMPAGNE (lag 14)'),
          Text(17, 0, 'BAGUETTE GRAINE (lag 7)'),
          Text(18, 0, 'SAVARIN (lag 7)'),
          Text(19, 0, 'FICELLE (lag 7)'),
          Text(20, 0, 'PARIS BREST (lag 14)'),
          Text(21, 0, 'BOULE 200G (lag 7)'),
          Text(22, 0, 'BANETTE (lag 14)'),
          Text(23, 0, 'PARIS BREST (lag 7)'),
          Text(24, 0, 'SAVARIN (lag 14)'),
          Text(25, 0, 'CAMPAGNE (lag 7)'),
          Text(26, 0, 'BOULE 400G (lag 7)'),
          Text(27, 0, 'BOULE 200G (lag 14)'),
          Text(28, 0, 'TARTE FRUITS 4P (lag 14)'),
          Text(29, 0, 'CEREAL BAGUETTE (lag 14)'),
          Text(30, 0, 'BRIOCHE (lag 14)'),
          Text(31, 0, 'BRIOCHE (lag 7)'),
          Text(32, 0, 'CEREAL BAGUETTE (lag 7)'),
          Text(33, 0, 'BOULE 400G (lag 14)'),
          Text(34, 0, 'TARTE FRUITS 4P (lag 7)'),
          Text(35, 0, 'FICELLE (lag 14)'),
          Text(36, 0, 'FRAISIER (lag 7)'),
          Text(37, 0, 'CHOU CHANTILLY (lag 7)'),
          Text(38, 0, 'PAIN AUX RAISINS (lag 14)'),
          Text(39, 0, 'BANETTINE (lag 14)'),
          Text(40, 0, 'PAIN BANETTE (lag 7)'),
          Text(41, 0, 'NOIX JAPONAISE (lag 7)'),
          Text(42, 0, 'KOUIGN AMANN (lag 7)'),
          Text(43, 0, 'MOISSON (lag 14)'),
          Text(44, 0, 'CAFE OU EAU (lag 2)'),
          Text(45, 0, 'TARTE FRUITS 6P (lag 7)'),
          Text(46, 0, 'BANETTINE (lag 7)'),
          Text(47, 0, 'MILLES FEUILLES (lag 14)'),
          Text(48, 0, 'PAIN AUX RAISINS (lag 7)'),
          Text(49, 0, 'NOIX JAPONAISE (lag 14)'),
          Text(50, 0, 'CAFE OU EAU (lag 9)'),
          Text(51, 0, 'KOUIGN AMANN (lag 14)'),
          Text(52, 0, 'FRAISIER (lag 14)'),
          Text(53, 0, 'MOISSON (lag 7)'),
          Text(54, 0, 'COMPLET (lag 14)'),
          Text(55, 0, 'CHOU CHANTILLY (lag 14)'),
          Text(56, 0, 'TARTE FRAISE 6P (lag 7)'),
          Text(57, 0, 'MILLES FEUILLES (lag 7)'),
          Text(58, 0, 'GRAND FAR BRETON (lag 7)'),
          Text(59, 0, 'TARTE FRAISE 4PER (lag 7)'),
```

```
 Text(60, 0, 'VIK BREAD (lag 14)'),
 Text(61, 0, 'TARTE FRUITS 6P (lag 14)'),
 Text(62, 0, 'TARTELETTE FRAISE (lag 7)'),
 Text(63, 0, 'FLAN (lag 7)'),
 Text(64, 0, 'GD KOUIGN AMANN (lag 7)'),
 Text(65, 0, 'TARTE FRAISE 6P (lag 14)'),
 Text(66, 0, 'PAIN BANETTE (lag 14)'),
 Text(67, 0, 'COMPLET (lag 7)'),
 Text(68, 0, 'GRAND FAR BRETON (lag 14)'),
 Text(69, 0, 'VIK BREAD (lag 7)'),
 Text(70, 0, 'DEMI BAGUETTE (lag 7)'),
 Text(71, 0, 'DEMI BAGUETTE (lag 14)'),
 Text(72, 0, 'FLAN ABRICOT (lag 7)'),
 Text(73, 0, 'BRIOCHETTE (lag 7)'),
 Text(74, 0, 'CROISSANT AMANDES (lag 14)')]
```



Check if these things are multicollinearity (VIF)

In [33]:
```python
import pandas as pd

# 1. Pivot to Wide Format
# Rows: Date, Columns: Products
wide_df = df.pivot(index='Date', columns='Product', values='Sales')
wide_df = wide_df.fillna(0)

# 2. Create Lags for all products
# Let's say you want Lags 1, 2, and 3 for every product
lags = [1, 2, 7, 14]
lagged_frames = []

for lag in lags:
    # Shift the entire dataframe and rename columns to 'Product_Lag_N'
    df_lagged = wide_df.shift(lag).add_suffix(f'_Lag{lag}')
    lagged_frames.append(df_lagged)

# Combine all lagged columns into one dataframe
wide_df_lags = pd.concat(lagged_frames, axis=1)

# 3. Clean up
```

```python
# Lags create NaN values at the start of the series; VIF will fail if the
wide_df_lags = wide_df_lags.dropna()

# Ensure all data is float for the VIF calculation
wide_df_lags = wide_df_lags.astype(float)

wide_df_lags
```

Out[33]:

Product Date	12 MACARON_Lag1	ARMORICAIN_Lag1	ARTICLE 295_Lag1	BAGUETTE_Lag1	BAGUE APERO_L
2021-01-16	0.0	0.0	0.0	17.1	
2021-01-17	0.0	0.0	0.0	25.2	
2021-01-18	0.0	0.0	0.0	33.3	
2021-01-19	0.0	0.0	0.0	24.3	
2021-01-20	0.0	0.0	0.0	18.0	
...	...	...	...	...	
2022-09-26	0.0	0.0	0.0	36.0	
2022-09-27	0.0	0.0	0.0	18.0	
2022-09-28	0.0	0.0	0.0	34.0	
2022-09-29	10.0	0.0	0.0	23.0	
2022-09-30	0.0	0.0	0.0	30.0	

623 rows × 592 columns

In [ ]:

```python
# Calculating VIF for each feature (No differencing)

if False:
    vif_data = pd.DataFrame()
    vif_data["feature"] = wide_df_lags.columns
    vif_data["VIF"] = [variance_inflation_factor(wide_df_lags.values, i)

    print(vif_data.sort_values(by="VIF", ascending=False).head(50))
```

```
/home/dantecz/Desktop/Projects/SteppingStone/.venv/lib/python3.8/site-pack
ages/statsmodels/regression/linear_model.py:1784: RuntimeWarning: invalid
value encountered in scalar divide
  return 1 - self.ssr/self.uncentered_tss
/home/dantecz/Desktop/Projects/SteppingStone/.venv/lib/python3.8/site-pack
ages/statsmodels/stats/outliers_influence.py:198: RuntimeWarning: divide b
y zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                          feature          VIF
400          PLATPREPARE6.00_Lag7          inf
70           GD PLATEAU SALE_Lag1          inf
251          PLATPREPARE5.50_Lag2          inf
462             BUCHE 6PERS_Lag14          inf
461             BUCHE 4PERS_Lag14          inf
142           TROIS CHOCOLAT_Lag1          inf
252          PLATPREPARE6.00_Lag2          inf
253          PLATPREPARE6.50_Lag2          inf
432      TARTELETTE COCKTAIL_Lag7          inf
256          PT PLATEAU SALE_Lag2          inf
407     REDUCTION SUCREES 24_Lag7          inf
404          PT PLATEAU SALE_Lag7          inf
392                    PATES_Lag7          inf
244                    PATES_Lag2          inf
386                PAIN NOIR_Lag7          inf
259     REDUCTION SUCREES 24_Lag2          inf
366          GD PLATEAU SALE_Lag7          inf
166              BUCHE 6PERS_Lag2          inf
165              BUCHE 4PERS_Lag2          inf
108          PT PLATEAU SALE_Lag1          inf
55      FORMULE PLAT PREPARE_Lag1          inf
96                     PATES_Lag1          inf
315              BUCHE 8PERS_Lag7          inf
314              BUCHE 6PERS_Lag7          inf
463             BUCHE 8PERS_Lag14          inf
313              BUCHE 4PERS_Lag7          inf
111     REDUCTION SUCREES 24_Lag1          inf
218          GD PLATEAU SALE_Lag2          inf
182  CRUMBLECARAMEL OU PISTAE_Lag2          inf
589                  TULIPE_Lag14          inf
90                 PAIN NOIR_Lag1          inf
192          DOUCEUR D HIVER_Lag2          inf
555    REDUCTION SUCREES 24_Lag14          inf
203     FORMULE PLAT PREPARE_Lag2          inf
552         PT PLATEAU SALE_Lag14          inf
106          PLATPREPARE7.00_Lag1          inf
167              BUCHE 8PERS_Lag2          inf
548         PLATPREPARE6.00_Lag14          inf
534               PAIN NOIR_Lag14          inf
547         PLATPREPARE5.50_Lag14          inf
103          PLATPREPARE5.50_Lag1          inf
18               BUCHE 6PERS_Lag1          inf
19               BUCHE 8PERS_Lag1          inf
105          PLATPREPARE6.50_Lag1          inf
17               BUCHE 4PERS_Lag1          inf
514         GD PLATEAU SALE_Lag14          inf
104          PLATPREPARE6.00_Lag1          inf
238                PAIN NOIR_Lag2          inf
178                    COUPE_Lag2  2334.973779
474                   COUPE_Lag14  2107.906254
```

In [ ]:

```
# Calculating VIF for each feature (No differencing)
if False:

    wide_df_lags_diff = wide_df_lags.diff().dropna()

    vif_diff_data = pd.DataFrame()
    vif_diff_data["feature"] = wide_df_lags_diff.columns
    vif_diff_data["VIF"] = [variance_inflation_factor(wide_df_lags_diff.v
```

```
print(vif_diff_data.sort_values(by="VIF", ascending=False).head(50))
```

```
/home/dantecz/Desktop/Projects/SteppingStone/.venv/lib/python3.8/site-pack
ages/statsmodels/regression/linear_model.py:1784: RuntimeWarning: invalid
value encountered in scalar divide
  return 1 - self.ssr/self.uncentered_tss
/home/dantecz/Desktop/Projects/SteppingStone/.venv/lib/python3.8/site-pack
ages/statsmodels/stats/outliers_influence.py:198: RuntimeWarning: divide b
y zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                          feature          VIF
400          PLATPREPARE6.00_Lag7          inf
70           GD PLATEAU SALE_Lag1          inf
462             BUCHE 6PERS_Lag14          inf
461             BUCHE 4PERS_Lag14          inf
432      TARTELETTE COCKTAIL_Lag7          inf
407     REDUCTION SUCREES 24_Lag7          inf
404          PT PLATEAU SALE_Lag7          inf
96                     PATES_Lag1          inf
111     REDUCTION SUCREES 24_Lag1          inf
392                    PATES_Lag7          inf
386                PAIN NOIR_Lag7          inf
55      FORMULE PLAT PREPARE_Lag1          inf
366          GD PLATEAU SALE_Lag7          inf
108          PT PLATEAU SALE_Lag1          inf
106          PLATPREPARE7.00_Lag1          inf
315              BUCHE 8PERS_Lag7          inf
203     FORMULE PLAT PREPARE_Lag2          inf
314              BUCHE 6PERS_Lag7          inf
313              BUCHE 4PERS_Lag7          inf
105          PLATPREPARE6.50_Lag1          inf
104          PLATPREPARE6.00_Lag1          inf
103          PLATPREPARE5.50_Lag1          inf
218          GD PLATEAU SALE_Lag2          inf
238                PAIN NOIR_Lag2          inf
259     REDUCTION SUCREES 24_Lag2          inf
256          PT PLATEAU SALE_Lag2          inf
90                 PAIN NOIR_Lag1          inf
253          PLATPREPARE6.50_Lag2          inf
252          PLATPREPARE6.00_Lag2          inf
251          PLATPREPARE5.50_Lag2          inf
463             BUCHE 8PERS_Lag14          inf
244                    PATES_Lag2          inf
552         PT PLATEAU SALE_Lag14          inf
534               PAIN NOIR_Lag14          inf
514         GD PLATEAU SALE_Lag14          inf
19               BUCHE 8PERS_Lag1          inf
182  CRUMBLECARAMEL OU PISTAE_Lag2          inf
555    REDUCTION SUCREES 24_Lag14          inf
18               BUCHE 6PERS_Lag1          inf
192          DOUCEUR D HIVER_Lag2          inf
17               BUCHE 4PERS_Lag1          inf
167              BUCHE 8PERS_Lag2          inf
142           TROIS CHOCOLAT_Lag1          inf
166              BUCHE 6PERS_Lag2          inf
547         PLATPREPARE5.50_Lag14          inf
165              BUCHE 4PERS_Lag2          inf
548         PLATPREPARE6.00_Lag14          inf
589                  TULIPE_Lag14          inf
258     REDUCTION SUCREES 12_Lag2  4986.979075
406     REDUCTION SUCREES 12_Lag7  4008.727499
```

In [ ]:

Out[ ]:

	feature	VIF
416	SAND JB_Lag7	11.196086
141	TRIANGLES_Lag1	11.111583
588	TROPEZIENNE FRAMBOISE_Lag14	11.001964
437	TRIANGLES_Lag7	10.858732
368	GRANDE SUCETTE_Lag7	10.744928
44	DOUCEUR D HIVER_Lag1	10.675538
350	FORMULE PATE_Lag7	10.621952
564	SAND JB_Lag14	10.450090
289	TRIANGLES_Lag2	10.372397
138	THE_Lag1	10.283342
394	PLAT_Lag7	10.252557
493	FINANCIER_Lag14	10.133225
424	SUCETTE_Lag7	10.033311
152	BAGUETTE APERO_Lag2	9.094387
390	PALMIER_Lag7	9.018178
286	THE_Lag2	8.752320
268	SAND JB_Lag2	8.379095
242	PALMIER_Lag2	8.315217
498	FORMULE PATE_Lag14	8.241478
488	DOUCEUR D HIVER_Lag14	8.144494
169	CAKE_Lag2	8.065837
434	THE_Lag7	7.953087
221	GUERANDAIS_Lag2	7.952693
300	BAGUETTE APERO_Lag7	7.657243
197	FINANCIER_Lag2	7.569754
144	TROPEZIENNE FRAMBOISE_Lag1	7.375107
448	BAGUETTE APERO_Lag14	7.313927
4	BAGUETTE APERO_Lag1	7.239838
118	SACHET DE VIENNOISERIE_Lag1	7.199366
292	TROPEZIENNE FRAMBOISE_Lag2	7.103859
440	TROPEZIENNE FRAMBOISE_Lag7	6.890157
54	FORMULE PATE_Lag1	6.862396
582	THE_Lag14	6.855742
266	SACHET DE VIENNOISERIE_Lag2	6.150517
465	CAKE_Lag14	5.841388
21	CAKE_Lag1	5.591784

	feature	VIF
510	GALETTE 8 PERS_Lag14	5.555677
202	FORMULE PATE_Lag2	5.553358
317	CAKE_Lag7	5.197188
94	PALMIER_Lag1	4.403172
2	ARTICLE 295_Lag1	NaN
99	PLAT 6.50E_Lag1	NaN
150	ARTICLE 295_Lag2	NaN
247	PLAT 6.50E_Lag2	NaN
298	ARTICLE 295_Lag7	NaN
395	PLAT 6.50E_Lag7	NaN
414	SACHET DE VIENNOISERIE_Lag7	NaN
446	ARTICLE 295_Lag14	NaN
543	PLAT 6.50E_Lag14	NaN
562	SACHET DE VIENNOISERIE_Lag14	NaN

In [34]:
```
# Calculate correlation matrix
corr_matrix = wide_df_lags.corr().abs()

# Look for pairs with correlation > 0.95
import numpy as np
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype
to_drop = [column for column in upper.columns if any(upper[column] > 0.95

print("Highly correlated pairs found:")
print(to_drop)
```

```
Highly correlated pairs found:
['BUCHE 6PERS_Lag1', 'GD PLATEAU SALE_Lag1', 'PLATPREPARE6.00_Lag1', 'REDU
CTION SUCREES 12_Lag1', 'REDUCTION SUCREES 24_Lag1', 'BUCHE 6PERS_Lag2', '
DOUCEUR D HIVER_Lag2', 'GD PLATEAU SALE_Lag2', 'PLATPREPARE6.00_Lag2', 'RE
DUCTION SUCREES 12_Lag2', 'REDUCTION SUCREES 24_Lag2', 'BUCHE 6PERS_Lag7',
'GD PLATEAU SALE_Lag7', 'PLATPREPARE6.00_Lag7', 'REDUCTION SUCREES 12_Lag7
', 'REDUCTION SUCREES 24_Lag7', 'BUCHE 6PERS_Lag14', 'BUCHE 8PERS_Lag14',
'GD PLATEAU SALE_Lag14', 'REDUCTION SUCREES 12_Lag14', 'REDUCTION SUCREES
24_Lag14', 'TULIPE_Lag14']
```

In [ ]:

Feature Engineering

Just making the lag features w/o split just in case

Make Lag features, One product

In [35]:
```
import pandas as pd
```

```
import numpy as np

def create_tabular_features(df, target_col, feature, date_col, lags=[1, 2
    """
    Converts time series into a supervised learning format.
    """
    # 1. Ensure the data is sorted by ID and Date to prevent shuffling la
    df = df.sort_values(by=[feature, date_col]).reset_index(drop=True)

    # 2. Create Lags (Target-1, Target-2, etc.)
    for lag in lags:
        df[f'lag_{lag}'] = df.groupby(feature)[target_col].shift(lag)

    # 3. Create Rolling Statistics (Moving Averages)
    # Note: We shift(1) first so we don't include the 'current' target in
    for window in [7, 30]:
        df[f'rolling_mean_{window}'] = (
            df.groupby(feature)[target_col]
            .transform(lambda x: x.shift(1).rolling(window=window).mean()
        )

    # 4. Extract Date-based features (The 'Time' context LightGBM needs)
    df[date_col] = pd.to_datetime(df[date_col])
    df['day_of_week'] = df[date_col].dt.dayofweek
    df['month'] = df[date_col].dt.month
    df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

    # 5. Drop rows with NaNs created by lagging (the 'cold start' problem
    return df.dropna().reset_index(drop=True)

# Example Usage:
# df_final = create_tabular_features(your_dataframe, 'sales', 'store_id',
```

In [36]:
```
### Not really relevant because it is making the whole dataset together,
```

In [37]:
```
df_final = create_tabular_features(df=df, target_col='Sales', feature='Pr

cat_cols = ['Product']
df_final[cat_cols] = df_final[cat_cols].astype('category').copy()
df_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53274 entries, 0 to 53273
Data columns (total 12 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Product          53274 non-null  category
 1   Date             53274 non-null  datetime64[ns]
 2   Sales            53274 non-null  float64
 3   unit_price       53274 non-null  float64
 4   lag_1            53274 non-null  float64
 5   lag_7            53274 non-null  float64
 6   lag_14           53274 non-null  float64
 7   rolling_mean_7   53274 non-null  float64
 8   rolling_mean_30  53274 non-null  float64
 9   day_of_week      53274 non-null  int32
 10  month            53274 non-null  int32
 11  is_weekend       53274 non-null  int64
dtypes: category(1), datetime64[ns](1), float64(7), int32(2), int64(1)
memory usage: 4.1 MB
```

In [38]:
```
df_final
```

Out[38]:

	Product	Date	Sales	unit_price	lag_1	lag_7	lag_14	rolling_mean_7
0	12 MACARON	2022-08-12	10.0	10.0	30.0	10.0	20.0	10.000000
1	12 MACARON	2022-08-13	20.0	10.0	10.0	0.0	10.0	10.000000
2	12 MACARON	2022-08-14	20.0	10.0	20.0	0.0	20.0	12.857143
3	12 MACARON	2022-08-15	10.0	10.0	20.0	10.0	0.0	15.714286
4	12 MACARON	2022-08-16	0.0	10.0	10.0	10.0	20.0	15.714286
...	...	...	...	...	...	...	...	...
53269	VIK BREAD	2022-09-26	0.0	2.7	27.0	0.0	0.0	11.185714
53270	VIK BREAD	2022-09-27	0.0	2.7	0.0	0.0	0.0	11.185714
53271	VIK BREAD	2022-09-28	0.0	2.7	0.0	0.0	0.0	11.185714
53272	VIK BREAD	2022-09-29	0.0	2.7	0.0	0.0	0.0	11.185714
53273	VIK BREAD	2022-09-30	32.4	2.7	0.0	24.3	21.6	11.185714

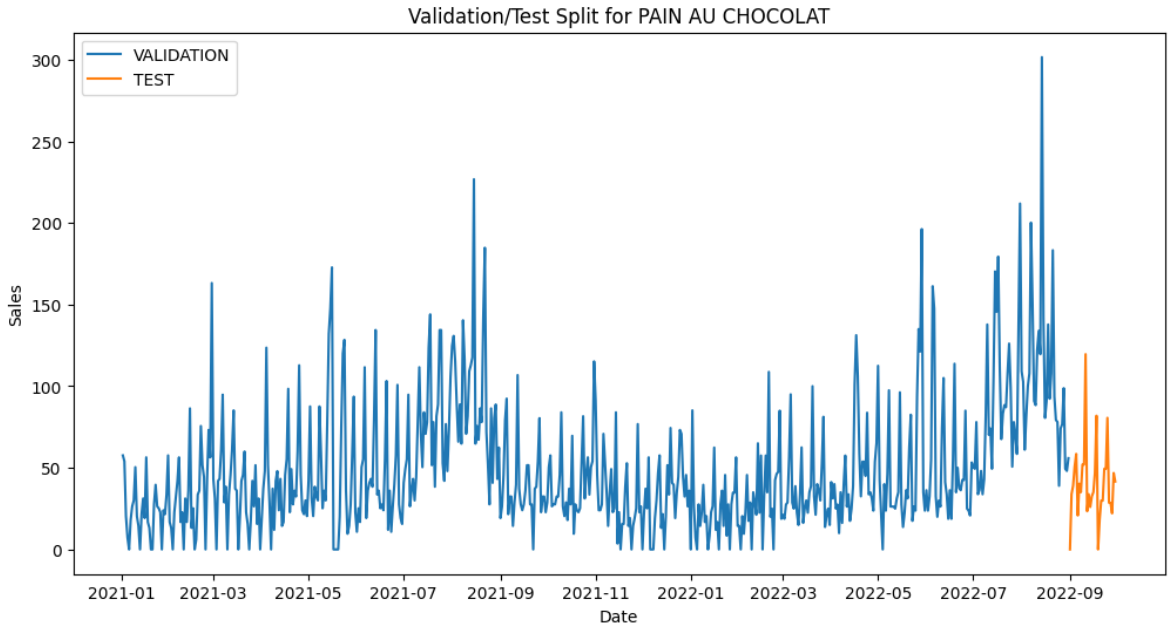
53274 rows × 12 columns

Split into validation and test sets

In [39]:
```
# ── Split on the RAW dataframe, before any feature engineering ──────────
# This prevents lag/rolling features from leaking future values into the

FORECAST_HORIZON_DAYS = 30          # hold out last N days as test
CUTOFF_DATE = df["Date"].max() - pd.Timedelta(days=FORECAST_HORIZON_DAYS)

df_validation_raw = df[df["Date"] <= CUTOFF_DATE].copy()
df_test_raw  = df[df["Date"] >  CUTOFF_DATE].copy()

print(f"Validation: {df_validation_raw['Date'].min()} → {df_validation_ra
print(f"Test : {df_test_raw['Date'].min()} → {df_test_raw['Date'].max()}


fig,ax = plt.subplots(figsize=(12,6))
prod = "PAIN AU CHOCOLAT"
mask = df_validation_raw["Product"] == prod
sns.lineplot(data=df_validation_raw.loc[mask], x="Date", y="Sales", ax=ax
mask = df_test_raw["Product"] == prod
sns.lineplot(data=df_test_raw.loc[mask], x="Date", y="Sales", ax=ax,label
ax.set_title(f"Validation/Test Split for {prod}", fontsize=12)
```

```
Validation: 2021-01-02 00:00:00 → 2022-08-31 00:00:00  (54,948 rows)
Test : 2022-09-01 00:00:00 → 2022-09-30 00:00:00  (2,098 rows)
```

Out[39]: Text(0.5, 1.0, 'Validation/Test Split for PAIN AU CHOCOLAT')



In [40]:
```
# Build features on train only; for test we append it to train so lags lo
# into the training period (avoids cold-start on the test window).

df_combined = pd.concat([df_validation_raw, df_test_raw]).sort_values(["P

df_featured = create_tabular_features(
    df=df_combined,
    target_col="Sales",
    feature="Product",
    date_col="Date",
    lags=[1, 7, 14],
)
```

```
df_featured["Product"] = df_featured["Product"].astype('category').copy()

# Re-split after feature engineering using the same cutoff
train = df_featured[df_featured["Date"] <= CUTOFF_DATE].copy()
test  = df_featured[df_featured["Date"] >  CUTOFF_DATE].copy()

TARGET   = "Sales"
CAT_COLS = ["Product"]
FEATURES = [c for c in train.columns if c not in [TARGET, "Date"]]

X_train, y_train = train[FEATURES], train[TARGET]
X_test,  y_test  = test[FEATURES],  test[TARGET]

print(f"Train shape: {X_train.shape}  |  Test shape: {X_test.shape}")
```

```
Train shape: (51182, 10)  |  Test shape: (2092, 10)
```

In [41]:

```
def objective(trial):
    params = {
        "objective":        "regression_l1",   # MAE-optimised
        "metric":           "mae",
        "verbosity":        -1,
        "boosting_type":    "gbdt",
        "n_estimators":     trial.suggest_int("n_estimators", 300, 2000),
        "learning_rate":    trial.suggest_float("learning_rate", 1e-3, 0.
        "num_leaves":       trial.suggest_int("num_leaves", 20, 300),
        "max_depth":        trial.suggest_int("max_depth", 3, 12),
        "min_child_samples":trial.suggest_int("min_child_samples", 5, 100
        "subsample":        trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5,
        "reg_alpha":        trial.suggest_float("reg_alpha", 1e-4, 10.0,
        "reg_lambda":       trial.suggest_float("reg_lambda", 1e-4, 10.0,
    }

    # Time-series CV: last 20 % of training data as internal validation
    val_cutoff = int(len(X_train) * 0.8)
    Xtr, Xval = X_train.iloc[:val_cutoff], X_train.iloc[val_cutoff:]
    ytr, yval = y_train.iloc[:val_cutoff], y_train.iloc[val_cutoff:]

    model = lgb.LGBMRegressor(**params)
    model.fit(
        Xtr, ytr,
        eval_set=[(Xval, yval)],
        callbacks=[lgb.early_stopping(50, verbose=False),
                   lgb.log_evaluation(-1)],
        categorical_feature=CAT_COLS,
    )
    preds = model.predict(Xval)
    return mean_absolute_error(yval, preds)


study = optuna.create_study(direction="minimize",
                            sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=50, show_progress_bar=True)

print(f"\nBest MAE (CV): {study.best_value:.4f}")
print("Best params:", study.best_params)
```

```
 0%|          | 0/50 [00:00<?, ?it/s]
```

```
Best MAE (CV): 11.7872
Best params: {'n_estimators': 567, 'learning_rate': 0.05937711121356566, '
num_leaves': 201, 'max_depth': 4, 'min_child_samples': 16, 'subsample': 0.
6389357165196756, 'colsample_bytree': 0.5007250058963767, 'reg_alpha': 0.0
6959945668016111, 'reg_lambda': 0.004765797927268501}
```

In [42]:

```
best_params = {
    "objective":     "regression_l1",
    "metric":        "mae",
    "verbosity":     -1,
    "boosting_type": "gbdt",
    **study.best_params,
}

final_model = lgb.LGBMRegressor(**best_params)
final_model.fit(
    X_train, y_train,
    categorical_feature=CAT_COLS,
)

print("Model trained ✓")
```

```
Model trained ✓
```

In [43]:

```
# FORECASTING WHILE CHEATING (using actual future lags from the test set)

y_pred_oneday = final_model.predict(X_test)
y_pred_oneday = np.clip(y_pred_oneday, 0, None)          # sales can't be

mae   = mean_absolute_error(y_test, y_pred_oneday)
rmse  = np.sqrt(mean_squared_error(y_test, y_pred_oneday))
# MAPE — skip zero-sales days to avoid division by zero
mask  = y_test > 0
mape  = np.mean(np.abs((y_test[mask] - y_pred_oneday[mask]) / y_test[mask

print(f"{'MAE':<8} {mae:.3f}")
print(f"{'RMSE':<8} {rmse:.3f}")
print(f"{'MAPE':<8} {mape:.2f}%  (non-zero actuals only)")
```

```
MAE      4.974
RMSE     11.986
MAPE     52.06%  (non-zero actuals only)
```

In [44]:

```
# FORECASTING RECURSIVELY (using predicted lags for the test set, which i

def recursive_forecast(model, train_df, test_df, features, target, cat_co
                       lags, windows):
    """
    Predicts one day at a time, feeding predictions back as lag inputs.
    Avoids using any true future sales values during forecasting.
    """
    # Working history starts as the full training set
    history = train_df.copy()

    predictions = []
    test_dates  = sorted(test_df["Date"].unique())

    for forecast_date in test_dates:
        # Rows we need to predict (one per product on this date)
        day_slice = test_df[test_df["Date"] == forecast_date].copy()
```

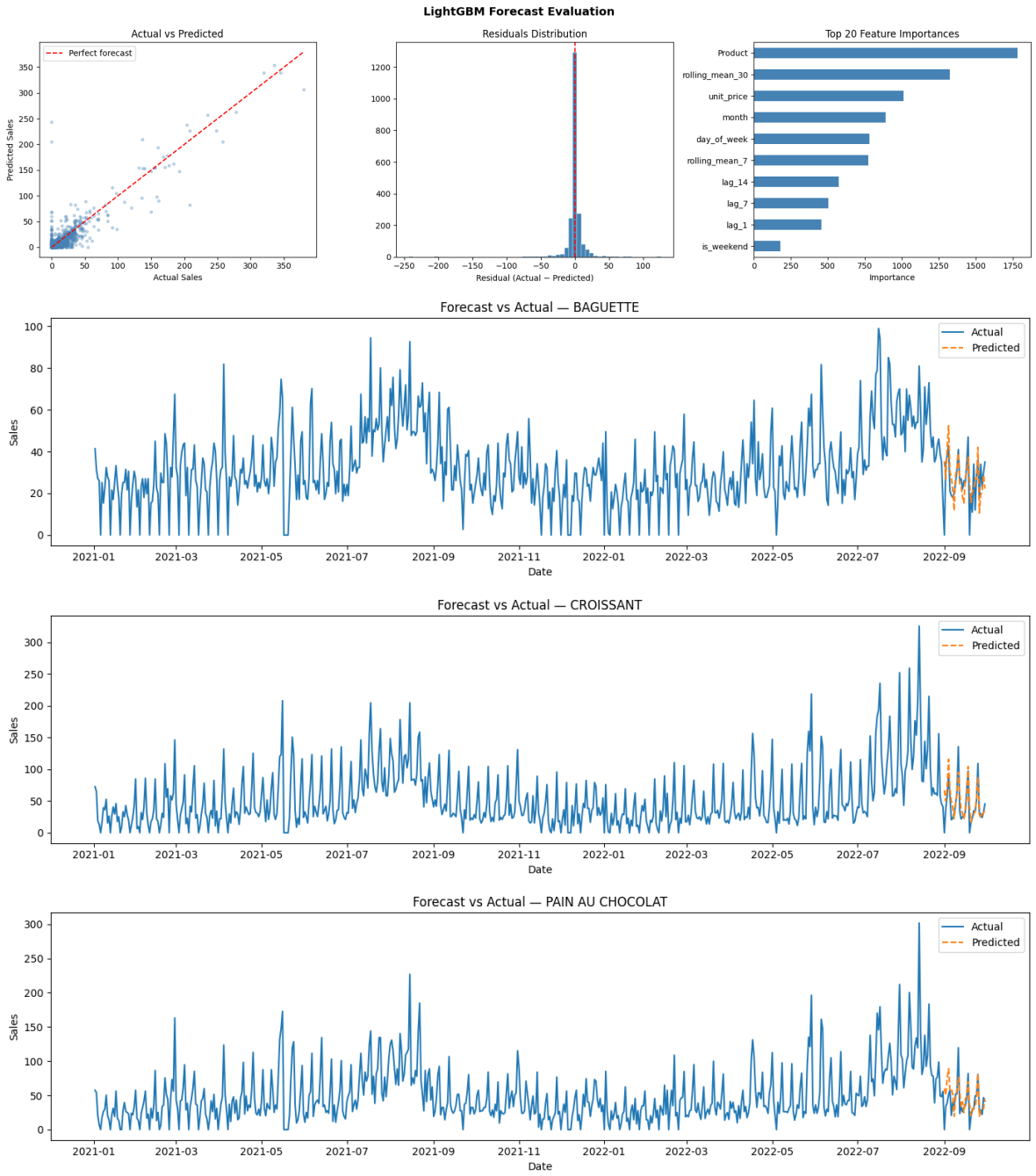
```
        # ── Rebuild lag & rolling features from history ──────────────
        for lag in lags:
            day_slice[f"lag_{lag}"] = day_slice["Product"].map(
                history.groupby("Product")[target].nth(-lag)   # nth from
            )

        for window in windows:
            day_slice[f"rolling_mean_{window}"] = day_slice["Product"].ma
                history.groupby("Product")[target]
                .apply(lambda x: x.iloc[-window:].mean())
            )

        # ── Predict ──────────────────────────────────────────────────
        X_day = day_slice[features]
        day_slice["prediction"] = np.clip(model.predict(X_day), 0, None)
        predictions.append(day_slice[["Product", "Date", target, "predict

        # ── Append predictions to history so next day's lags are correct
        # We substitute the true Sales with our prediction so no future l
        pseudo_row = day_slice.copy()
        pseudo_row[target] = pseudo_row["prediction"]
        history = pd.concat([history, pseudo_row], ignore_index=True)

    return pd.concat(predictions, ignore_index=True)


LAGS    = [1, 7, 14]
WINDOWS = [7, 30]

results = recursive_forecast(
    model       = final_model,
    train_df    = train,          # engineered train set
    test_df     = test,           # engineered test set (features will be
    features    = FEATURES,
    target      = TARGET,
    cat_cols    = CAT_COLS,
    lags        = LAGS,
    windows     = WINDOWS,
)

# ── Metrics ──────────────────────────────────────────────────────────
mae  = mean_absolute_error(results[TARGET], results["prediction"])
rmse = np.sqrt(mean_squared_error(results[TARGET], results["prediction"])
mask = results[TARGET] > 0
mape = np.mean(np.abs((results.loc[mask, TARGET] - results.loc[mask, "pre
                      / results.loc[mask, TARGET])) * 100

print(f"{'MAE':<8} {mae:.3f}")
print(f"{'RMSE':<8} {rmse:.3f}")
print(f"{'MAPE':<8} {mape:.2f}%  (non-zero actuals only)")

y_pred_recursive = results["prediction"]
y_pred_recursive
```

```
MAE      9.998
RMSE     28.076
MAPE     89.28%  (non-zero actuals only)
```

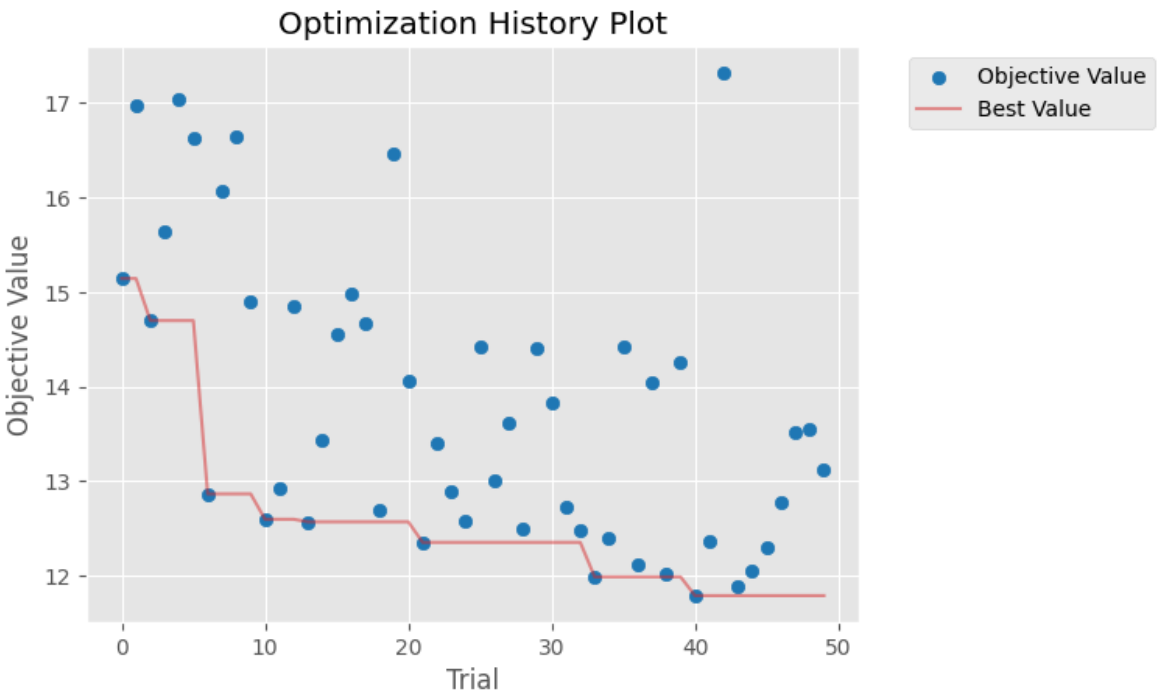
Out[44]:
```
0       1.537521
1       3.726196
2       0.046115
3       6.200250
4       0.633958
          ...
2087    0.648942
2088    0.022830
2089    0.867112
2090    0.748566
2091    0.000000
Name: prediction, Length: 2092, dtype: float64
```

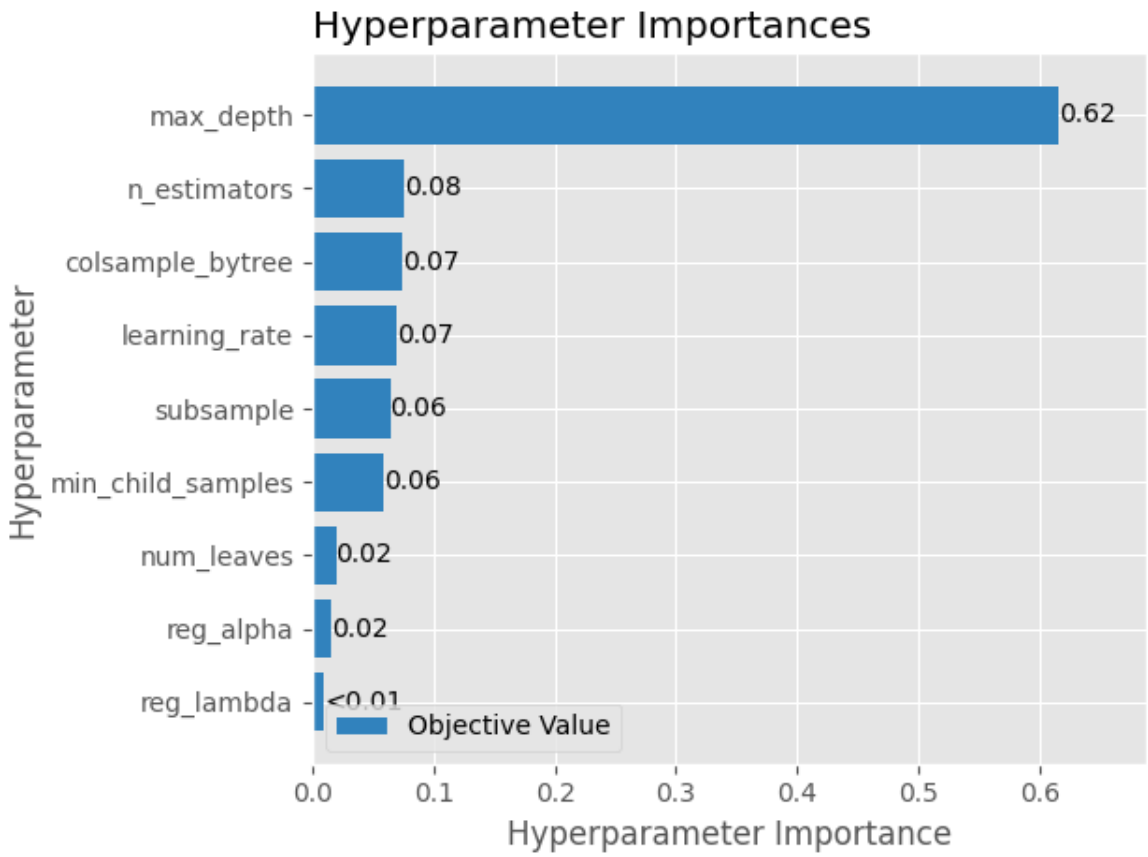
In [45]:
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("LightGBM Forecast Evaluation", fontsize=14, fontweight="bol

y_pred = y_pred_oneday


# ── 1. Actual vs Predicted scatter ──────────────────────────────────
ax = axes[0]
ax.scatter(y_test, y_pred, alpha=0.3, s=10, color="steelblue")
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
ax.plot(lims, lims, "r--", lw=1.5, label="Perfect forecast")
ax.set_xlabel("Actual Sales"); ax.set_ylabel("Predicted Sales")
ax.set_title("Actual vs Predicted"); ax.legend()

# ── 2. Residuals distribution ───────────────────────────────────────
ax = axes[1]
residuals = y_test.values - y_pred
ax.hist(residuals, bins=60, color="steelblue", edgecolor="white", linewid
ax.axvline(0, color="red", lw=1.5, linestyle="--")
ax.set_xlabel("Residual (Actual − Predicted)")
ax.set_title("Residuals Distribution")

# ── 3. Feature importance (top 20) ──────────────────────────────────
ax = axes[2]
fi = pd.Series(final_model.feature_importances_, index=FEATURES).nlargest
fi.sort_values().plot(kind="barh", ax=ax, color="steelblue")
ax.set_title("Top 20 Feature Importances")
ax.set_xlabel("Importance")

plt.tight_layout()
plt.show()

# ── 4. Time-series plot for one product ─────────────────────────────
for prod in ["BAGUETTE","CROISSANT","PAIN AU CHOCOLAT"]:
#     sample_product = test["Product"].value_counts().index[i]
    sample_product = prod
    mask_p = test["Product"] == sample_product

    plt.figure(figsize=(14, 4))
#     plt.plot(test.loc[mask_p, "Date"], y_test[mask_p.values],
#              label="Actual", lw=1.5)
    plt.plot(df.loc[df["Product"] == sample_product, "Date"], df.loc[df["
             label="Actual", lw=1.5)

    plt.plot(test.loc[mask_p, "Date"], y_pred[mask_p.values],
             label="Predicted", lw=1.5, linestyle="--")
    plt.title(f"Forecast vs Actual — {sample_product}")
```

```
    plt.xlabel("Date"); plt.ylabel("Sales")
    plt.legend(); plt.tight_layout(); plt.show()
```



In [46]:
```
from optuna.visualization.matplotlib import (
    plot_optimization_history,
    plot_param_importances,
)

# fig, axes = plt.subplots(1, 2, figsize=(14, 5))
plot_optimization_history(study)
plot_param_importances(study)
plt.tight_layout()
plt.show()
```





Now look at a single time series!

In [58]:
```
import pmdarima as pm                    # pip install pmdarima
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.seasonal import seasonal_decompose, STL
from statsmodels.tsa.statespace.sarimax import SARIMAX
from scipy.signal import periodogram
import matplotlib.gridspec as gridspec
from scipy import stats
```

```
plt.rcParams.update({"figure.dpi": 120, "axes.spines.top": False,
                     "axes.spines.right": False})
```

In [59]:
```
df[df["Product"] == "BAGUETTE"]
```

Out[59]:

	Product	Date	Sales	Price
Index_Date				
2021-01-02	BAGUETTE	2021-01-02	41.4	0.9
2021-01-03	BAGUETTE	2021-01-03	31.5	0.9
2021-01-04	BAGUETTE	2021-01-04	27.0	0.9
2021-01-05	BAGUETTE	2021-01-05	26.1	0.9
2021-01-06	BAGUETTE	2021-01-06	0.0	0.9
...	...	...	...	...
2022-09-26	BAGUETTE	2022-09-26	18.0	1.0
2022-09-27	BAGUETTE	2022-09-27	34.0	1.0
2022-09-28	BAGUETTE	2022-09-28	23.0	1.0
2022-09-29	BAGUETTE	2022-09-29	30.0	1.0
2022-09-30	BAGUETTE	2022-09-30	35.0	1.0

637 rows × 4 columns

Cell 2 — Isolate series & basic cleaning

In [60]:
```
dfb = df[df["Product"] == "BAGUETTE"][["Date", "Sales", "Price"]].copy()
# dfb = dfb.rename(columns={"Price": "Price"})
dfb["Date"] = pd.to_datetime(dfb["Date"])

# ── Ensure daily frequency (no gaps) ──────────────────────────────
full_range = pd.date_range(dfb["Date"].min(), dfb["Date"].max(), freq="D"
dfb = dfb.set_index("Date").reindex(full_range)
dfb.index.name = "Date"

# ── Fill missings ─────────────────────────────────────────────────
dfb["Sales"] = dfb["Sales"].fillna(0)          # missing day = no sales
dfb["Price"] = dfb["Price"].ffill()             # carry forward last know

# ── Dtypes ────────────────────────────────────────────────────────
dfb["Sales"] = dfb["Sales"].astype(float)
dfb["Price"] = dfb["Price"].astype(float)

print(dfb.dtypes)
print(f"\nDate range : {dfb.index.min().date()} → {dfb.index.max().date()
print(f"Total days : {len(dfb)}")
print(f"Null Sales : {dfb['Sales'].isna().sum()}")
print(f"Null Price : {dfb['Price'].isna().sum()}")
dfb.head(10)
```

```
Sales    float64
Price    float64
dtype: object

Date range : 2021-01-02 → 2022-09-30
Total days : 637
Null Sales : 0
Null Price : 0
```

Out[60]:

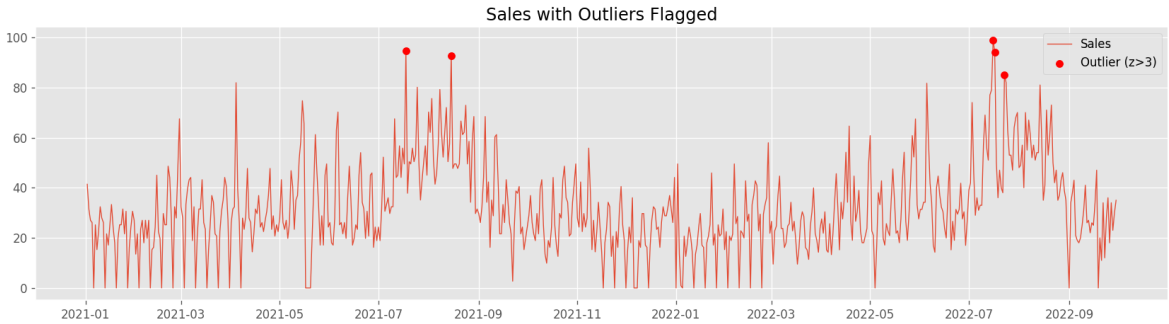
Date	Sales	Price
2021-01-02	41.4	0.9
2021-01-03	31.5	0.9
2021-01-04	27.0	0.9
2021-01-05	26.1	0.9
2021-01-06	0.0	0.9
2021-01-07	25.2	0.9
2021-01-08	15.3	0.9
2021-01-09	22.5	0.9
2021-01-10	32.4	0.9
2021-01-11	27.9	0.9

Cell 3 — Outlier detection & treatment

In [65]:

```
# ── Z-score method (flag > 3 SD from mean) ──────────────────────────
# Purpose: identify data-entry errors or exceptional events that would
# distort model training. We cap rather than drop to preserve daily conti

z_scores = np.abs(stats.zscore(dfb["Sales"]))
outlier_mask = z_scores > 3 # Change this to something larger like 4, her

print(f"Outliers detected (|z| > 3): {outlier_mask.sum()}")
print(dfb[outlier_mask][["Sales", "Price"]])

# ── IQR method (second opinion) ──────────────────────────────────────
Q1, Q3 = dfb["Sales"].quantile([0.25, 0.75])
IQR    = Q3 - Q1
iqr_mask = (dfb["Sales"] < Q1 - 3 * IQR) | (dfb["Sales"] > Q3 + 3 * IQR)
print(f"\nOutliers detected (IQR × 3): {iqr_mask.sum()}")

# ── Visualise ────────────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(14, 4))
ax.plot(dfb.index, dfb["Sales"], lw=0.8, label="Sales")
ax.scatter(dfb.index[outlier_mask], dfb.loc[outlier_mask, "Sales"],
           color="red", zorder=5, label="Outlier (z>3)")
ax.set_title("Sales with Outliers Flagged"); ax.legend()
plt.tight_layout(); plt.show()

# ── Winsorise: cap at 99th percentile ────────────────────────────────
cap = dfb["Sales"].quantile(0.99)
dfb["Sales_clean"] = dfb["Sales"].clip(upper=cap)
print(f"\nCap applied at 99th pct: {cap:.1f}")
```

```
Outliers detected (|z| > 3): 5
            Sales  Price
Date
2021-07-18   94.5    0.9
2021-08-15   92.7    0.9
2022-07-16   99.0    1.0
2022-07-17   94.0    1.0
2022-07-23   85.0    1.0

Outliers detected (IQR × 3): 0
```




```
Cap applied at 99th pct: 81.8
```

Cell 4 — Exploratory plots

In [75]:
```
# Purpose: Get an intuitive feel for the series before any modelling.
# Look for: trend, seasonality, price-sales relationship, distributional

fig = plt.figure(figsize=(16, 12))
gs  = gridspec.GridSpec(3, 2, figure=fig, hspace=0.45, wspace=0.3)

# 1. Sales over time
ax1 = fig.add_subplot(gs[0, :])
ax1.plot(dfb.index, dfb["Sales_clean"], lw=0.9, color="steelblue")
ax1b = ax1.twinx()
ax1b.plot(dfb.index, dfb["Price"], color="darkorange", lw=0.8, alpha=0.6)
ax1b.set_ylabel("Price", color="darkorange")
ax1.set_title("Daily Sales (blue) & Price (orange)")
ax1.set_ylabel("Sales")

# 2. Day-of-week box plot — reveals weekly seasonality
ax2 = fig.add_subplot(gs[1, 0])
dfb["dow"] = dfb.index.dayofweek
dfb.boxplot(column="Sales_clean", by="dow", ax=ax2)
ax2.set_xticklabels(["Mon","Tue","Wed","Thu","Fri","Sat","Sun"])
ax2.set_title("Sales by Day of Week")
ax2.set_xlabel(""); plt.sca(ax2); plt.title("Sales by Day of Week")

# 3. Monthly box plot — reveals annual seasonality
ax3 = fig.add_subplot(gs[1, 1])
dfb["month"] = dfb.index.month
dfb.boxplot(column="Sales_clean", by="month", ax=ax3)
ax3.set_title("Sales by Month")
ax3.set_xlabel("Month"); plt.sca(ax3); plt.title("Sales by Month")

# 4. Sales distribution
ax4 = fig.add_subplot(gs[2, 0])
dfb["Sales_clean"].hist(bins=40, ax=ax4, color="steelblue", edgecolor="wh
ax4.axvline(dfb["Sales_clean"].median(), color="red", linestyle="--", lab
ax4.set_title("Sales Distribution"); ax4.legend()
```

```
# 5. Price vs Sales scatter
ax5 = fig.add_subplot(gs[2, 1])
ax5.scatter(dfb["Price"], dfb["Sales_clean"], alpha=0.3, s=10, color="ste
ax5.set_xlabel("Price"); ax5.set_ylabel("Sales")
ax5.set_title("Price vs Sales")

plt.suptitle("Baguette — Exploratory Analysis", fontsize=13, fontweight="
plt.show()
```

Cell 5 — Stationarity tests

In [67]:

```
# ── ADF & KPSS tests ──────────────────────────────────────────────
# PURPOSE:
#   SARIMAX requires a stationary series (constant mean/variance over tim
#   ADF (Augmented Dickey-Fuller): H0 = unit root (non-stationary).
#     → p < 0.05: reject H0, evidence of stationarity.
#   KPSS: H0 = stationary.
#     → p < 0.05: reject H0, evidence of non-stationarity.
#   Use both together — they test opposite nulls, so agreement gives conf
#   If non-stationary → differencing (d parameter in ARIMA) is needed.

def stationarity_report(series, label="Series"):
    print(f"{'-'*50}")
    print(f"  {label}")
    print(f"{'-'*50}")

    adf_stat, adf_p, _, _, adf_cv, _ = adfuller(series.dropna(), autolag=
    print(f"  ADF  stat={adf_stat:.3f}  p={adf_p:.4f}  "
          f"→ {'STATIONARY ✓' if adf_p < 0.05 else 'NON-STATIONARY ✗'}")
    print(f"       Critical values: { {k: round(v,3) for k,v in adf_cv.it

    kpss_stat, kpss_p, _, kpss_cv = kpss(series.dropna(), regression="c",
    print(f"  KPSS stat={kpss_stat:.3f}  p={kpss_p:.4f}  "
```

```
            f"→ {'STATIONARY ✓' if kpss_p >= 0.05 else 'NON-STATIONARY ✗'}"
    print()

stationarity_report(dfb["Sales_clean"], "Sales (raw)")

# If non-stationary, check first difference
dfb["Sales_diff1"] = dfb["Sales_clean"].diff()
stationarity_report(dfb["Sales_diff1"].dropna(), "Sales (1st difference)"
```

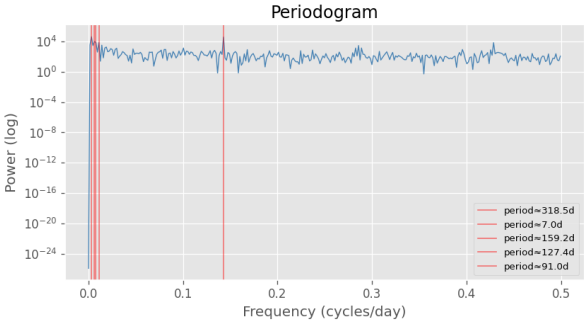
```
────────────────────────────────────────────────────
  Sales (raw)
────────────────────────────────────────────────────
  ADF  stat=-2.440  p=0.1307  → NON-STATIONARY ✗
       Critical values: {'1%': -3.441, '5%': -2.866, '10%': -2.569}
  KPSS stat=0.386  p=0.0833  → STATIONARY ✓

────────────────────────────────────────────────────
  Sales (1st difference)
────────────────────────────────────────────────────
  ADF  stat=-6.502  p=0.0000  → STATIONARY ✓
       Critical values: {'1%': -3.441, '5%': -2.866, '10%': -2.569}
  KPSS stat=0.037  p=0.1000  → STATIONARY ✓
```

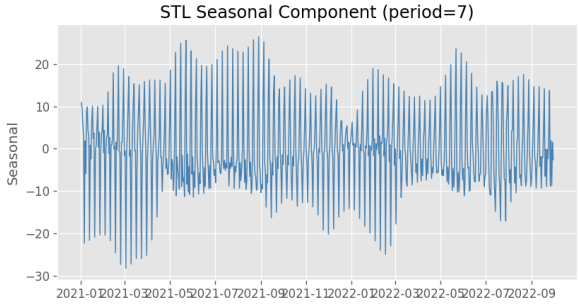
Cell 6 — Seasonality detection

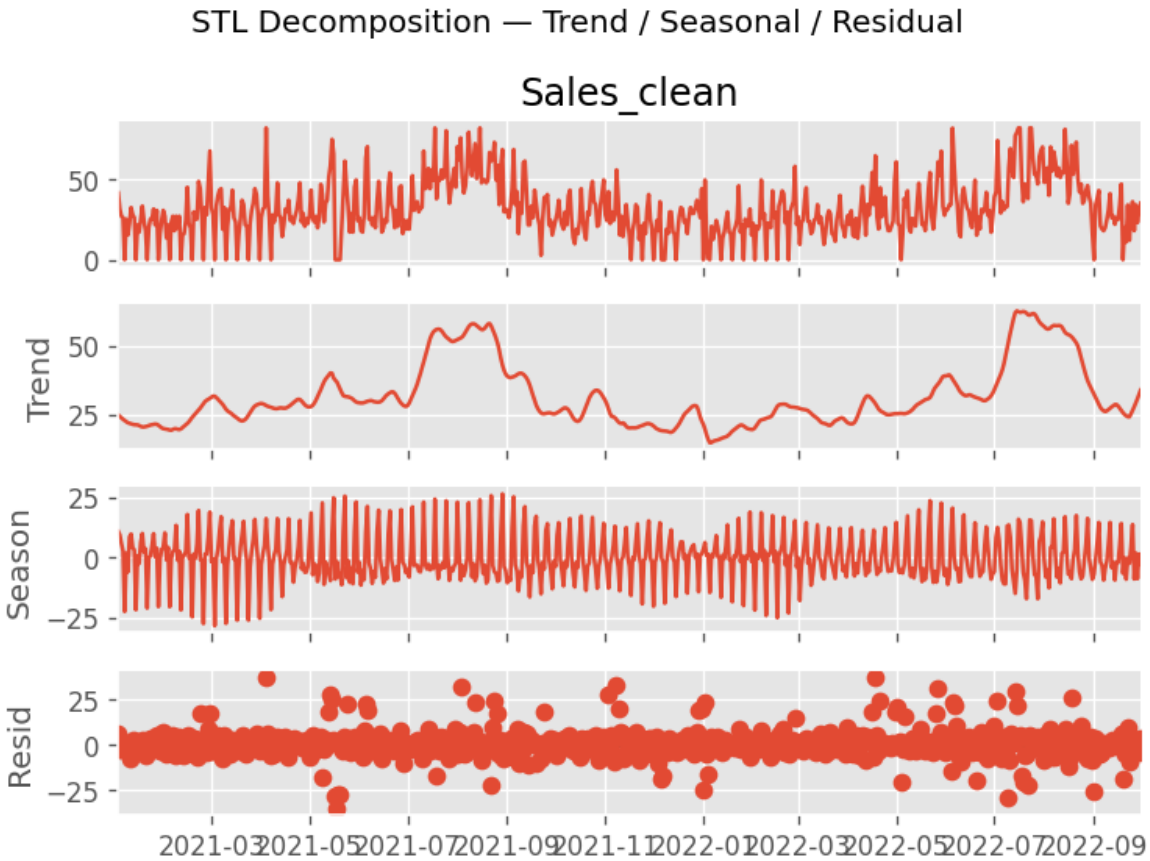
In [68]:
```
# ── 1. Periodogram — find dominant cycles ──────────────────────────
# PURPOSE: Converts the time series into the frequency domain. Peaks in t
# periodogram indicate repeating cycles. The x-axis shows cycles-per-day,
# so 1/7 ≈ 0.143 = weekly cycle, 1/30 ≈ 0.033 = monthly cycle.

freqs, power = periodogram(dfb["Sales_clean"].fillna(0))
dominant_idx = np.argsort(power)[::-1][:5]

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].semilogy(freqs, power, color="steelblue", lw=0.8)
for idx in dominant_idx:
    if freqs[idx] > 0:
        axes[0].axvline(freqs[idx], color="red", alpha=0.5, lw=1,
                        label=f"period≈{1/freqs[idx]:.1f}d")
axes[0].set_xlabel("Frequency (cycles/day)"); axes[0].set_ylabel("Power (
axes[0].set_title("Periodogram"); axes[0].legend(fontsize=8)

print("Top 5 dominant periods (days):")
for idx in dominant_idx:
    if freqs[idx] > 0:
        print(f"  Period ≈ {1/freqs[idx]:.1f} days  (power={power[idx]:.1

# ── 2. STL Decomposition ──────────────────────────────────────────
# PURPOSE: Separates the series into Trend + Seasonal + Residual componen
# Helps confirm the period found above and see if seasonality is stable.
# A large seasonal component relative to residual = strong, modelable sea

PERIOD = 7   # update if periodogram suggests otherwise

stl = STL(dfb["Sales_clean"], period=PERIOD, robust=True)
res = stl.fit()

axes[1].plot(dfb.index, res.seasonal, lw=0.9, color="steelblue")
```

```
axes[1].set_title(f"STL Seasonal Component (period={PERIOD})")
axes[1].set_ylabel("Seasonal")
plt.tight_layout(); plt.show()

res.plot()
plt.suptitle("STL Decomposition — Trend / Seasonal / Residual", fontsize=
plt.tight_layout(); plt.show()
```

```
Top 5 dominant periods (days):
  Period ≈ 318.5 days  (power=42935.3)
  Period ≈ 7.0 days  (power=38553.6)
  Period ≈ 159.2 days  (power=10070.0)
  Period ≈ 127.4 days  (power=7576.5)
  Period ≈ 91.0 days  (power=7322.4)
```



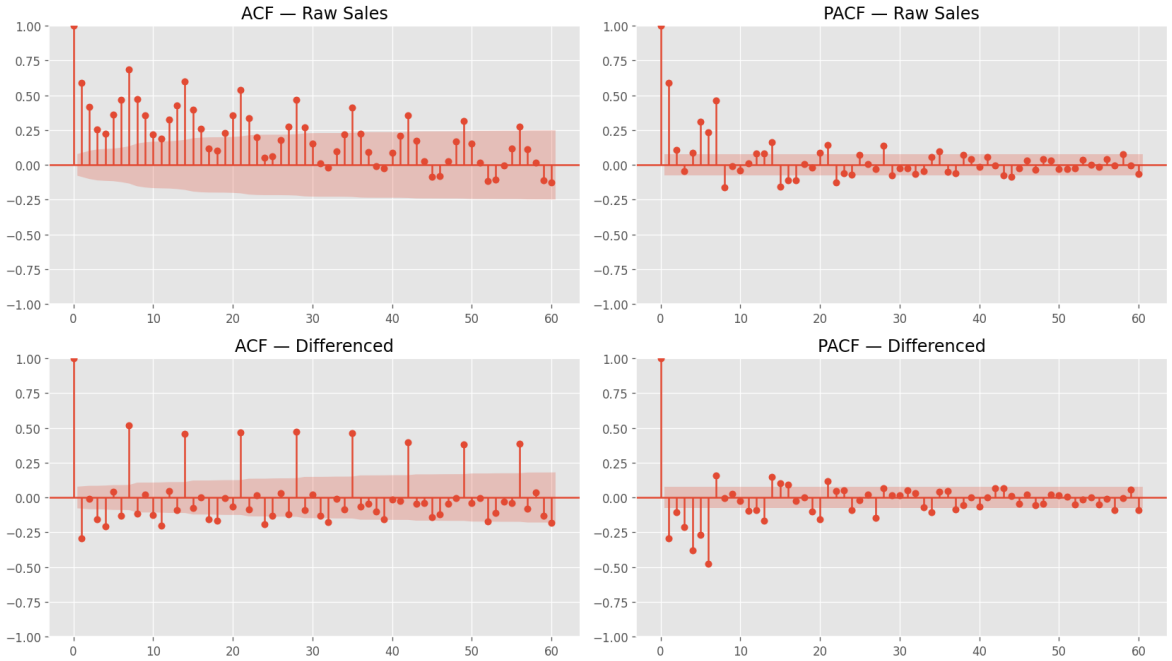




Cell 7 — ACF & PACF

In [69]:
```
# PURPOSE:
#   ACF (Autocorrelation Function): correlation of the series with its ow
#     → Tells us the MA(q) order: cut-off after lag q suggests MA(q).
#     → Spikes at multiples of 7 confirm weekly seasonality.
#   PACF (Partial ACF): correlation at lag k after removing shorter-lag e
#     → Tells us the AR(p) order: cut-off after lag p suggests AR(p).
#   Together these guide the (p,d,q)(P,D,Q,s) parameter choices for SARIM
```

```
#   though auto_arima will search these automatically in the next cell.

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

plot_acf( dfb["Sales_clean"],          lags=60, ax=axes[0,0], title="ACF —
plot_pacf(dfb["Sales_clean"],          lags=60, ax=axes[0,1], title="PACF
plot_acf( dfb["Sales_diff1"].dropna(),lags=60, ax=axes[1,0], title="ACF —
plot_pacf(dfb["Sales_diff1"].dropna(),lags=60, ax=axes[1,1], title="PACF

plt.tight_layout(); plt.show()
```



Cell 8 — Train/test split

In [71]:
```
TEST_DAYS = 30

train_s = dfb.iloc[:-TEST_DAYS]
test_s  = dfb.iloc[-TEST_DAYS:]

y_train = train_s["Sales_clean"]
y_test  = test_s["Sales_clean"]
X_train = train_s[["Price"]]
X_test  = test_s[["Price"]]

print(f"Train: {y_train.index[0].date()} → {y_train.index[-1].date()}  ({
print(f"Test : {y_test.index[0].date()}  → {y_test.index[-1].date()}  ({l
```

```
Train: 2021-01-02 → 2022-08-31  (607 days)
Test : 2022-09-01  → 2022-09-30  (30 days)
```

Cell 9 — Auto SARIMAX fit

In [72]:
```
# auto_arima performs a stepwise search over (p,d,q)(P,D,Q,s) combination
# minimising AIC. We pass Price as an exogenous regressor (X).
# m=7 tells it to consider weekly seasonality.

auto_model = pm.auto_arima(
    y_train,
    X              = X_train,
    m              = 7,              # weekly seasonal period
```

```
    seasonal       = True,
    d              = None,           # let it determine differencing orde
    D              = None,           # let it determine seasonal differen
    stepwise       = True,           # faster than exhaustive search
    information_criterion = "aic",
    trace          = True,           # print candidate models
    error_action   = "ignore",
    suppress_warnings = True,
)

print("\n", auto_model.summary())
```

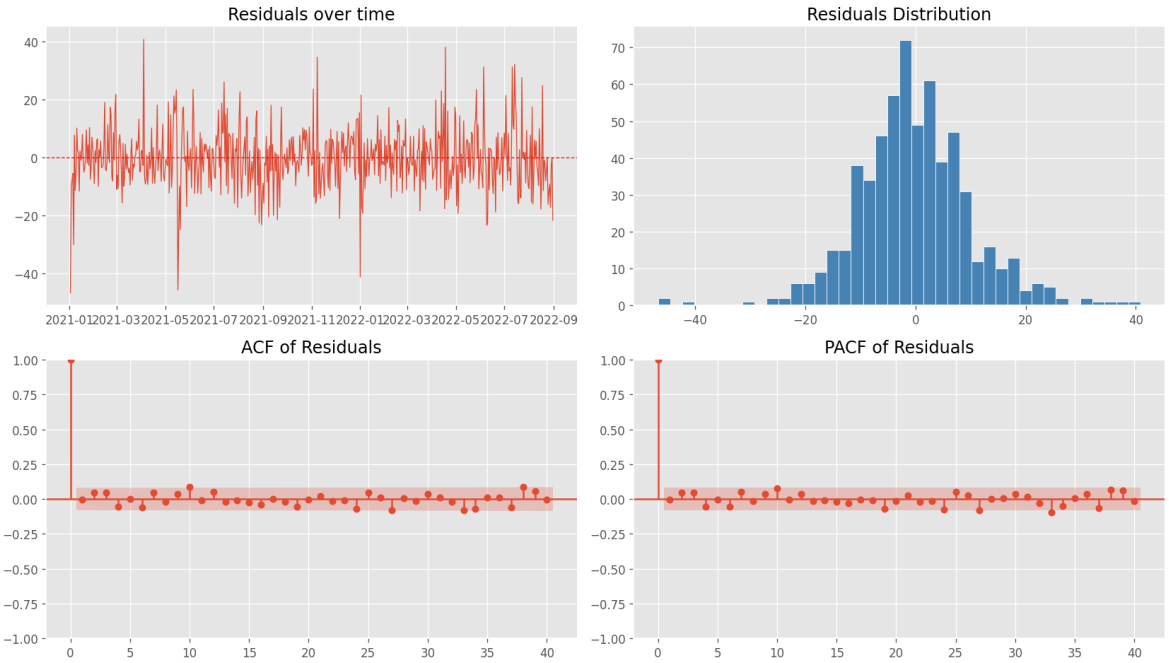
```
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(1,0,1)[7] intercept   : AIC=4575.707, Time=2.00 sec
 ARIMA(0,1,0)(0,0,0)[7] intercept   : AIC=5054.883, Time=0.03 sec
 ARIMA(1,1,0)(1,0,0)[7] intercept   : AIC=4802.863, Time=0.30 sec
 ARIMA(0,1,1)(0,0,1)[7] intercept   : AIC=4816.476, Time=0.68 sec
 ARIMA(0,1,0)(0,0,0)[7]             : AIC=5052.884, Time=0.09 sec
 ARIMA(2,1,2)(0,0,1)[7] intercept   : AIC=inf, Time=1.70 sec
 ARIMA(2,1,2)(1,0,0)[7] intercept   : AIC=4692.837, Time=1.25 sec
 ARIMA(2,1,2)(2,0,1)[7] intercept   : AIC=4576.469, Time=5.32 sec
 ARIMA(2,1,2)(1,0,2)[7] intercept   : AIC=4576.573, Time=3.58 sec
 ARIMA(2,1,2)(0,0,0)[7] intercept   : AIC=4865.208, Time=0.46 sec
 ARIMA(2,1,2)(0,0,2)[7] intercept   : AIC=inf, Time=3.72 sec
 ARIMA(2,1,2)(2,0,0)[7] intercept   : AIC=4649.509, Time=4.10 sec
 ARIMA(2,1,2)(2,0,2)[7] intercept   : AIC=4578.839, Time=4.89 sec
 ARIMA(1,1,2)(1,0,1)[7] intercept   : AIC=4573.501, Time=1.24 sec
 ARIMA(1,1,2)(0,0,1)[7] intercept   : AIC=4769.149, Time=0.67 sec
 ARIMA(1,1,2)(1,0,0)[7] intercept   : AIC=inf, Time=0.81 sec
 ARIMA(1,1,2)(2,0,1)[7] intercept   : AIC=4573.360, Time=4.25 sec
 ARIMA(1,1,2)(2,0,0)[7] intercept   : AIC=4628.080, Time=4.56 sec
 ARIMA(1,1,2)(2,0,2)[7] intercept   : AIC=4575.866, Time=6.39 sec
 ARIMA(1,1,2)(1,0,2)[7] intercept   : AIC=4573.328, Time=5.37 sec
 ARIMA(1,1,2)(0,0,2)[7] intercept   : AIC=4708.754, Time=4.50 sec
 ARIMA(0,1,2)(1,0,2)[7] intercept   : AIC=4575.654, Time=4.32 sec
 ARIMA(1,1,1)(1,0,2)[7] intercept   : AIC=4572.678, Time=4.73 sec
 ARIMA(1,1,1)(0,0,2)[7] intercept   : AIC=4723.074, Time=2.19 sec
 ARIMA(1,1,1)(1,0,1)[7] intercept   : AIC=4572.515, Time=1.69 sec
 ARIMA(1,1,1)(0,0,1)[7] intercept   : AIC=4767.160, Time=0.53 sec
 ARIMA(1,1,1)(1,0,0)[7] intercept   : AIC=4673.350, Time=1.28 sec
 ARIMA(1,1,1)(2,0,1)[7] intercept   : AIC=4572.717, Time=5.11 sec
 ARIMA(1,1,1)(0,0,0)[7] intercept   : AIC=4884.932, Time=0.14 sec
 ARIMA(1,1,1)(2,0,0)[7] intercept   : AIC=4629.244, Time=3.11 sec
 ARIMA(1,1,1)(2,0,2)[7] intercept   : AIC=4575.250, Time=4.74 sec
 ARIMA(0,1,1)(1,0,1)[7] intercept   : AIC=4588.326, Time=0.63 sec
 ARIMA(1,1,0)(1,0,1)[7] intercept   : AIC=4650.017, Time=0.37 sec
 ARIMA(2,1,1)(1,0,1)[7] intercept   : AIC=4573.464, Time=0.91 sec
 ARIMA(0,1,0)(1,0,1)[7] intercept   : AIC=4737.459, Time=0.27 sec
 ARIMA(0,1,2)(1,0,1)[7] intercept   : AIC=4575.049, Time=1.42 sec
 ARIMA(2,1,0)(1,0,1)[7] intercept   : AIC=4618.936, Time=0.64 sec
 ARIMA(1,1,1)(1,0,1)[7]             : AIC=4550.282, Time=0.73 sec
 ARIMA(1,1,1)(0,0,1)[7]             : AIC=4766.141, Time=0.23 sec
 ARIMA(1,1,1)(1,0,0)[7]             : AIC=4671.463, Time=0.48 sec
 ARIMA(1,1,1)(2,0,1)[7]             : AIC=inf, Time=2.71 sec
 ARIMA(1,1,1)(1,0,2)[7]             : AIC=4553.890, Time=3.05 sec
 ARIMA(1,1,1)(0,0,0)[7]             : AIC=4867.285, Time=0.23 sec
 ARIMA(1,1,1)(0,0,2)[7]             : AIC=4704.730, Time=2.04 sec
 ARIMA(1,1,1)(2,0,0)[7]             : AIC=4626.276, Time=2.91 sec
 ARIMA(1,1,1)(2,0,2)[7]             : AIC=4572.703, Time=3.99 sec
 ARIMA(0,1,1)(1,0,1)[7]             : AIC=4569.363, Time=0.96 sec
 ARIMA(1,1,0)(1,0,1)[7]             : AIC=4648.020, Time=0.28 sec
 ARIMA(2,1,1)(1,0,1)[7]             : AIC=4550.896, Time=0.76 sec
 ARIMA(1,1,2)(1,0,1)[7]             : AIC=4553.250, Time=0.92 sec
 ARIMA(0,1,0)(1,0,1)[7]             : AIC=4734.425, Time=0.36 sec
 ARIMA(0,1,2)(1,0,1)[7]             : AIC=4553.463, Time=0.80 sec
 ARIMA(2,1,0)(1,0,1)[7]             : AIC=4616.937, Time=0.43 sec
 ARIMA(2,1,2)(1,0,1)[7]             : AIC=4565.717, Time=1.05 sec

Best model:  ARIMA(1,1,1)(1,0,1)[7]
Total fit time: 109.975 seconds

                                     SARIMAX Results
```

```
==============================================================================
===============
Dep. Variable:                                y   No. Observations:
607
Model:             SARIMAX(1, 1, 1)x(1, 0, 1, 7)   Log Likelihood
-2269.141
Date:                           Sat, 25 Apr 2026   AIC
4550.282
Time:                                   17:45:54   BIC
4576.723
Sample:                               01-02-2021   HQIC
4560.570
                                    - 08-31-2022
Covariance Type:                             opg
==============================================================================
====
                 coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------
----
Price         97.9010    112.467      0.870      0.384    -122.530     31
8.332
ar.L1          0.2602      0.040      6.525      0.000       0.182
0.338
ma.L1         -0.8562      0.021    -40.469      0.000      -0.898
-0.815
ar.S.L7        0.9907      0.005    199.366      0.000       0.981
1.000
ma.S.L7       -0.8714      0.025    -34.489      0.000      -0.921
-0.822
sigma2       102.7489      4.382     23.450      0.000      94.161     11
1.337
==============================================================================
=========
Ljung-Box (L1) (Q):                   0.06   Jarque-Bera (JB):
84.59
Prob(Q):                              0.81   Prob(JB):
0.00
Heteroskedasticity (H):               1.19   Skew:
0.22
Prob(H) (two-sided):                  0.23   Kurtosis:
4.78
==============================================================================
=========

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (com
plex-step).
```

Cell 10 — Residual diagnostics

In [73]:
```
# PURPOSE: Check that model residuals look like white noise.
#   If patterns remain, the model has missed structure.
#   Ljung-Box test H0 = no autocorrelation in residuals.
#   → p > 0.05 at all lags = residuals are clean.

residuals = pd.Series(auto_model.resid(), index=y_train.index)

fig, axes = plt.subplots(2, 2, figsize=(14, 8))
```

```
axes[0,0].plot(residuals, lw=0.8); axes[0,0].set_title("Residuals over ti
axes[0,0].axhline(0, color="red", lw=0.8, linestyle="--")

residuals.hist(bins=40, ax=axes[0,1], color="steelblue", edgecolor="white
axes[0,1].set_title("Residuals Distribution")

plot_acf(residuals, lags=40, ax=axes[1,0], title="ACF of Residuals")
plot_pacf(residuals, lags=40, ax=axes[1,1], title="PACF of Residuals")

plt.tight_layout(); plt.show()

lb = acorr_ljungbox(residuals, lags=[7, 14, 21, 28], return_df=True)
print("Ljung-Box test (p > 0.05 = residuals are white noise):")
print(lb)
```



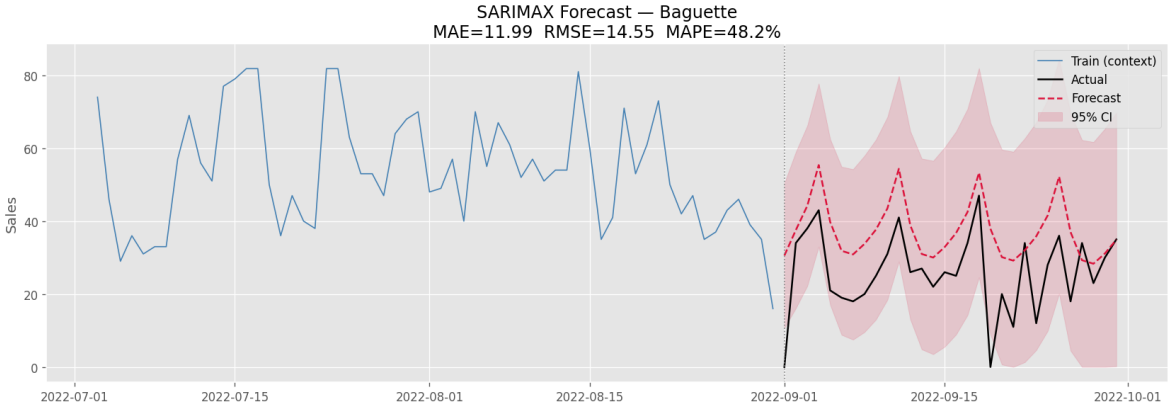

```
Ljung-Box test (p > 0.05 = residuals are white noise):
      lb_stat  lb_pvalue
7    7.558319   0.373145
14  15.026817   0.376324
21  18.892524   0.592035
28  27.928504   0.468241
```

Cell 11 — Forecast & evaluate

In [74]:
```
forecast, conf_int = auto_model.predict(
    n_periods = TEST_DAYS,
    X         = X_test,
    return_conf_int = True,
)
forecast   = np.clip(forecast, 0, None)
forecast_s = pd.Series(forecast,   index=y_test.index)
lower      = pd.Series(np.clip(conf_int[:, 0], 0, None), index=y_test.ind
upper      = pd.Series(conf_int[:, 1],                   index=y_test.ind

# ── Metrics ──────────────────────────────────────────────────────
mae   = mean_absolute_error(y_test, forecast_s)
rmse  = np.sqrt(mean_squared_error(y_test, forecast_s))
mask  = y_test > 0
mape  = np.mean(np.abs((y_test[mask] - forecast_s[mask]) / y_test[mask]))
```

```
print(f"{'MAE':<8} {mae:.3f}")
print(f"{'RMSE':<8} {rmse:.3f}")
print(f"{'MAPE':<8} {mape:.2f}%  (non-zero actuals only)")

# ── Plot ──────────────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(14, 5))

# Show last 60 days of training for context
context = y_train.iloc[-60:]
ax.plot(context.index,    context,      color="steelblue", lw=1, label="T
ax.plot(y_test.index,     y_test,       color="black",     lw=1.5, label=
ax.plot(forecast_s.index, forecast_s,   color="crimson",   lw=1.5, linest
ax.fill_between(forecast_s.index, lower, upper, color="crimson", alpha=0.
ax.axvline(y_test.index[0], color="grey", linestyle=":", lw=1)

ax.set_title(f"SARIMAX Forecast — Baguette\nMAE={mae:.2f}  RMSE={rmse:.2f
ax.set_ylabel("Sales"); ax.legend()
plt.tight_layout(); plt.show()
```

```
MAE      11.991
RMSE     14.548
MAPE     48.18%  (non-zero actuals only)
```



In [ ]: